Implementation guide, worked example

Omega Memory and Goals in MeTTaDinC

WMTM's Open-Item Store, from Journal to Gate

Summary statistics, receipts, a changepoint monitor and a promotion gate on eight simulated weeks of the goal engine

Companion to Part II, *Live Memory through Summary Statistics*, to Part I, *A Practical Implementation Guide*, and to the MeTTaDinC specification draft

4 November 2026

Developed from the d-calculus and Hyperseed project

Scope. Part II ended with a protocol of eight steps for applying its method to one live store. This document carries the protocol out for the open items of WMTM's goal engine, from declaring the consumers to reporting the claims at a promotion gate. The data are a simulated deployment: eight weeks of the engine's log-only phase, generated from a fixed seed, in which a parser update degrades reply detection in the fourth week and the team detects, confirms and repairs it. Section 7 checks the store against the next release, which adds a snooze action and a three-day follow-up goal.

Validation material. The modules ship as one file in a bundle with the simulation, the twelve frozen streams it wrote, a registry patch for the one form this example adds, Z3 encodings of every symbolic request, Part II's reference evaluator extended by that form, and an oracle that regenerates the deployment, recomputes the numbers of the text and answers every request from the module text (99 checks). Where this text and the bundle differ, the bundle is authoritative. No data from a running WMTM system were used, and nothing here describes a change already made to one.

Abstract

Part II of this guide showed how to state the memory decisions of a live store as summary statistics, certificates that a solver checks for every state at once, and how to measure from logged receipts what no declaration can settle. This worked example applies the method end to end to one store, the open items from which WMTM's goal engine proposes follow-ups, along the eight steps of Part II's protocol. The data are simulated: eight weeks of the engine's log-only phase, with 1,357 items, 955 replies, a reply detector that misses some of them, spurious closing notes, paraphrased questions, and two record designs running side by side.

The declarations take three short modules. A solver establishes the event statistic and the store's indexes for every timing and every store size, results reported as Conditional on that solver because its proofs are not yet checked; the window design, which closes items by what a recall window shows, is Refuted with replayable witnesses; and a replay of the frozen journal through the module's maps agrees with the simulated designs on all 8,966 daily decisions. The receipts needed more care than the protocol gave them. The engine proposes an open item every day, so one missed reply yields a run of stale proposals, and the example reduces the labels to replies, keys, key lifetimes, closings and audited items, each stream naming its population, selection rule and policy.

A parser update on day 21 raises the detector's miss rate from 1.5 to 25 percent. Part II's mixture monitor, which accumulates evidence from the first receipt, never fires: three healthy weeks count as evidence for the accepted rate. A Shiryaev–Roberts changepoint monitor fires 47 hours after the update; replies audited after the alarm confirm the regression, and a rollback and re-parse restore all 18 lost replies. At the promotion gate, four claims from the reviewed receipts establish at level 0.01 that fewer than one reply in twenty is followed by a stale proposal, that fewer than one key in twenty is a duplicate and one closing in twenty is false, and that the event design errs on fewer items than the window design. The random audit that checks the review behind those receipts is still too small to establish its own two claims, and the gate promotes the engine on conditions it records with the claims. The next release makes the deployed engine index invalid input to the checker before any data are seen; its replacement is the coarsest the consumers allow, and every index can be rebuilt from the durable records.

How to read this example

Section 1 says what the example is and why its data are simulated. Section 2 describes the store, the goal engine and the simulated eight weeks. The protocol follows in the order of the calendar: Section 3 declares the store (Steps 1 to 4 of Part II's protocol), Section 4 designs and freezes the receipts (Steps 5 and 6), Section 5 runs the monitor through the parser regression (Step 8), and Section 6 reports the claims at the promotion gate (Step 7) and answers the question Part II left to this example. Section 7 checks the store against the next release, Section 8 collects what the example teaches implementers, and Sections 9 and 10 state the limits and conclude. The appendices hold the module file, the registry patch, the frozen streams and the reproduction notes.

A reader who wants the outcome first can read Section 2, then Sections 5.4 and 6.4. Paragraphs headed *In plain terms* restate a result without notation. "Part II" is the second part of this guide [1], "Part I" the first [2], and "the specification" the MeTTaDinC specification draft [3]. Days are counted from the start of the deployment; day 0 is a Monday.

Contents

1 Introduction **4**
1.1 What the example adds to Part II . . . 4
1.2 Status . . . 5

2 The deployment and its simulation **5**
2.1 WMTM's open items and the goal engine . . . 5
2.2 The simulation . . . 6
2.3 The eight weeks . . . 7

3 Declaring the store **7**
3.1 Step 1: the consumers . . . 8
3.2 Step 2: the item machine and the abstraction function . . . 8
3.3 Step 3: the statistics and the deployed window . . . 9
3.4 Step 4: the store, durable records and indexes . . . 11
3.5 Conformance replay . . . 12

4 Receipts and streams **12**
4.1 Why single proposals are the wrong unit . . . 13
4.2 Step 5: five units of receipt . . . 13
4.3 Where the labels come from . . . 15
4.4 Step 6: freezing the receipts . . . 15

5 The regression **17**
5.1 Step 8: the monitors . . . 17
5.2 What happened . . . 18
5.3 Why a monitor started at the first receipt stays silent . . . 18
5.4 A changepoint monitor . . . 19
5.5 Confirming and repairing . . . 20

6 The promotion gate **21**
6.1 What the gate decides . . . 21
6.2 Step 7: the claims . . . 22
6.3 Reading the results . . . 22
6.4 The decision . . . 23
6.5 Planning the audit . . . 24
6.6 The gate's record . . . 24
6.7 The confidence budget . . . 25
6.8 The question Part II left to the example . . . 25

7 The next release **26**
7.1 The item with a snooze . . . 26
7.2 The deployed engine index is invalid input . . . 27
7.3 The record and the engine index after the release . . . 27
7.4 The three-day nag . . . 29
7.5 Rebuilding the indexes from the records . . . 30
7.6 What the release brings to the gate . . . 31

8 What the example teaches implementers **32**
8.1 Declarations . . . 32
8.2 Receipts . . . 32
8.3 Monitors . . . 33

8.4 Gates 33
8.5 Releases and durable records 33

9 Limits **33**

10 Conclusion **34**

References **34**

A The module file **36**

B The registry patch **41**

C The frozen streams **42**

D Reproducing the results **43**

1 Introduction

Part II of this guide carried Part I's finite models of Omega's memory and goals over to the live system, where ages are real numbers, sources form an open set and stores grow without bound. Its central object was the summary statistic: an encoding of the live state, with an abstract update for every operation and a read map for every query, whose defining identities a solver checks for all states at once. What no declaration can settle, such as how often the detector that decides whether a message is a reply misses one, was to be estimated from logged receipts with confidence sequences and watched with monitors that stay valid however often they are read. Part II closed with a protocol of eight steps for applying all this to one store, and named the store: WMTM's open items and the goal engine, whose first live review prompted Part I [1, Section 9].

This document carries the protocol out. What I wanted from it was to see the method at the size of a deployment, next to a journal and a review log, including the morning a monitor fires, and to find out which parts of the protocol needed more than Part II gave them.

The data are simulated. The live journal, the engine's suggestion log and the labels of a live review were not available for this work. A seeded simulation stands in for them. It is built from what Parts I and II and the WMTM report say about the store and the engine [5], and its parameters were chosen to be plausible rather than measured from a running system. Every number in this text comes from that simulation. The simulation writes its output once as frozen streams pinned by SHA-256 digests, the module reads only the streams, and the oracle regenerates them byte for byte from the seed. Running the same requests on streams frozen from the live system needs no change to the declarations except the digests.

What happens in the eight weeks. During the engine's log-only phase, its proposals are logged and reviewed rather than shown in the prompt. Two record designs run side by side on the same journal: the recall window that the engine used at its first review, and the event statistic that Part II recommends. For three weeks nothing unusual happens. On the Monday of the fourth week a parser update lowers the reply detector's accuracy, and the share of replies it misses rises from 1.5 to 25 percent. A changepoint monitor on the reply receipts raises an alarm on the Wednesday morning, 47 hours after the update. On the Thursday morning the team audits the replies received since the update; those received after the alarm confirm it, so the team rolls the parser back at noon and re-parses the journal, which restores all 18 replies the update had lost. The monitor restarts the next morning and stays quiet for the rest of the eight weeks. At the end of the eighth week a promotion gate decides, from the receipts of the last four weeks, whether the engine's suggestions go into the prompt with the event design. After the gate, the next release adds a snooze action and a three-day follow-up goal, and the store is checked against it before it ships.

1.1 What the example adds to Part II

Most of the protocol works as Part II wrote it. Five points needed more than the protocol said.

1. Receipts have to be reduced to units whose average is what a claim is about. The engine proposes an open item every day, so one missed reply produces up to seven stale proposals of the same item, and in the log-only phase the number of proposals depends on a policy that promotion changes. Part II warned about this [1, Section 5.2]; the example replaces per-proposal labels by five kinds of receipt, one per reply, per key, per key lifetime, per closing and per audited item, and each stream names its population, selection rule and policy (Section 4).

2. A monitor whose evidence accumulates from the first receipt reacts late to a change that comes late. Part II's mixture monitor never fires on the regression, because the healthy weeks before it count as evidence for the accepted rate. The example proposes a changepoint form of the same monitors, which needs two new arguments of one request, bounds its false alarms over any fixed horizon, and confirms its alarm on replies that the alarm did not use (Section 5).
3. Every claim that the gate can establish from receipts rests on a review whose blind spots have to be named. The review of proposals cannot see false closings, so the example adds a review of closings; and the random audit, which checks the review independently, collects observations at a third and a sixth of the rate of the reply receipts and is not yet decisive after four weeks. Because the intervals are confidence sequences, the gate can report those claims now and again later without spending more of its confidence budget (Section 6).
4. A release that adds an action makes the deployed engine index invalid input to the checker before any data are seen, and the coarsest replacement merges into one class every open item that a snooze has pushed past the engine's seven days (Section 7).
5. Keeping the event statistic in the durable records pays for itself at the first goal change. The rebuild of every index from the records is itself a summary statistic, which the solver establishes (Section 7.5).

The example also answers the question Part II left to it: how much of the stale-suggestion rate is due to the record design, which the certified statistics remove, and how much to missed replies, which only better detection reduces (Section 6.8).

1.2 Status

The three modules of the example conform to the registry together with the patches of the worked examples [4], of Part I, of Part II, and of this example, which adds two arguments to one request. The verdict of every symbolic request has been obtained twice: by the cvc5 solver [19], on the obligations that Part II's reference evaluator builds from the module text, and by the Z3 solver [18], on encodings written by hand; for a refuted request, both solvers find a witness. No proof of unsatisfiability is checked, so under the specification's assurance policy these verdicts are Conditional on cvc5, the solver the reference evaluator calls, with Z3's agreement recorded beside them [3, Section 2.2], [1, Section 7.2]; the tables say Conditional for them. Verdicts computed in exact arithmetic are Checked, and refutations are Refuted with witnesses replayed by direct evaluation. Every estimate and monitor has been computed from the frozen streams by the evaluator and by the Python models, and the evaluator refuses a stream whose content no longer matches its declared digest; the oracle tests this on a copy of the receipts with every label changed. No MeTTaDinC runtime exists yet. The deployment is simulated, and nothing here describes a change made to a running Omega or WMTM system.

2 The deployment and its simulation

2.1 WMTM's open items and the goal engine

WMTM (Working Medium-Term Memory) is the memory and reasoning layer of the Proto-Cosmo2 agent. It keeps a journal of notes, and some notes carry explicit markers. A note that begins "open question for Ben:" or "pending:" opens an item; a note marked "decided:" that answers it closes it [5]. Deciding whether a note answers an open item is the job of a reply detector in the parser, which matches the note to the item's person and topic. Each item is

filed under a deduplication key, the pair of the person it concerns and a topic label that the parser assigns.

The goal engine reads this store once a day. It proposes a follow-up for every item that is open and at most seven days old, once per key. Whether its proposals reach the language model's prompt is controlled by a flag, `goal_prompt` in Part I's notes [2, Section 4.5]. While the flag is off, in the *log-only phase*, the proposals are written to a log, and a reviewer reads the log and labels each proposal: stale if the item had already been answered, duplicate if another key holds the same question. The reviewer also reads the thread of the first closing note that memory records for each item and labels it spurious if the note did not answer the item. Turning the flag on is a change to the running agent, and under GOLEM-Iter, the framework under which such changes are tested and promoted, it passes a *promotion gate*: a decision point at which the claims the change rests on are checked or estimated and recorded [6].

Other readers use the same store. The agent's commitment record asks whether an item is open, whatever its age. A follow-up goal, here called the nag goal, asks whether an item is open and at most two days old, so that the agent can follow up while the question is fresh. These readers are the store's consumers, and what the store must remember is decided by them.

The first live review of the engine failed partly because "stale items stayed open (old replies were missed)" and "paraphrases repeated" [5]. Part I traced the first failure, in a finite model, to closing items by what the engine's recall window shows: a reply that has left the window makes an answered item look open again. Part II showed the same on real time [1, Section 6.1]. The log-only phase of this example therefore runs two record designs side by side on the same journal. Under the *window design*, the first closing note the engine has learned of counts for two days from the time it was learned, and the item looks open again afterwards; this is how Part I modeled the engine at its first review. Under the *event design*, the item is closed from its first closing note on. Both designs decide on every key at every daily cycle, and both proposal lists are logged and reviewed.

2.2 The simulation

The simulation generates the journal one item at a time. Table 1 lists its parameters. Items are opened at a Poisson rate of 30 a day on weekdays and 8 a day at weekends, at times spread uniformly over working hours, from 08:00 to 20:00. Each concerns one of six people, with Ben the most frequent at about one item in three, and gets a topic label made of one of 25 topic stems and the item's number. Seventy-two percent of items receive a reply, after a delay drawn from a lognormal distribution with a median of 0.8 days; the spread is wide, so some replies come within the hour and some after more than a week, at any hour of the day.

Three kinds of input error corrupt what memory sees. The reply detector misses a reply with probability 0.015 under parser version 1 and 0.25 under version 2, which is deployed on day 21 at 10:00. A spurious closing note, a "decided:" note that the detector attaches to the wrong item, arrives for 1.2 percent of items after an exponentially distributed delay with a mean of 1.5 days. A paraphrase, a later note that asks the same question under a new topic label, occurs for 0.6 percent of items within a day of the original, and it creates a second key. A third of the items, chosen at random, are audited at the age of four days by a person who reads the whole thread and records whether a reply had arrived and whether memory shows the item as closed.

The engine's cycle runs at 09:00 every day. At each cycle both designs decide on every key that is at most seven days old, and the review labels every proposal of either design and the first closing of every key that memory learned of since the last cycle. Once the review is done, the changepoint monitor of Section 5.4 reads one receipt for each reply received since the previous cycle, in the order of the replies' times. When it fires, the team follows a rule fixed in

advance (Section 5.5): the next morning it audits the replies received since the update, and if those received after the alarm show a miss rate above the accepted one, it rolls the parser back at noon and re-parses the journal written since the update. Each reply that version 2 missed is then recovered with the probability that version 1 finds a reply. The monitor restarts at the first cycle after the rollback.

Parameter	Value	Note
Length of the deployment	56 days	day 0 is a Monday
Items opened per day	30, weekends 8	Poisson; 08:00 to 20:00
People	6	Ben about one item in three
Share of items with a reply	0.72	
Delay to the reply	lognormal	median 0.8 days, log-scale $\sigma = 1.1$
Detector misses a reply	0.015; 0.25	parser v1; v2 from day 21, 10:00
Spurious closing notes	0.012 of items	delay exponential, mean 1.5 days
Paraphrased questions	0.006 of items	new topic label within 0.05 to 1 day
Audited items	one in three	at age 4 days
Engine's cycle	09:00 daily	items at most 7 days old
Recall window of the window design	2 days	
Accepted rate of stale reply receipts	1/20	alternatives 1/10, 1/5, 2/5
Alarm threshold of the monitor	100,000	mean time to false alarm
Level of estimates and confirmation	0.01	
Gate window	days 28 to 55	receipts released on these days
Seed	20261104	

Table 1: Parameters of the simulated deployment.

The simulation keeps the ground truth, which replies arrived and which keys are duplicates, beside the parsed journal. It uses the truth only to write the review's labels and the audit's findings and to check the estimates afterwards; neither record design sees it. Two simplifications should be kept in mind. The simulated review and audit make no mistakes, so every label is correct, which in a live deployment is a hypothesis that the audit checks (Section 6.4). The input errors are independent of each other by construction. The gate's one-sided claims need neither independence nor a common rate (Section 4.1), but a two-sided interval read as an estimate of one rate, and the planning sizes of Section 6.5, assume both, and here they hold by design. Neither the language model nor the content of the notes is modeled, since no consumer of this store reads them.

2.3 The eight weeks

Table 2 gives the calendar. Over the eight weeks 1,357 items were opened, 1,218 of them on weekdays, and the journal of the eight weeks holds 2,326 notes. The items produced 1,362 keys, five of them duplicates created by paraphrases. Before day 56, 955 items received a reply; nine received a spurious closing note; 438 were audited at age four. The event design logged 3,767 proposals, 67.3 a day on average, and the window design 7,176.

3 Declaring the store

This section takes the first four steps of Part II's protocol: declare the consumers, write the item machine and the abstraction function, declare and check the statistics against the record design in use, and lift them to the store.

When	What happens
Days 0 to 20	Log-only phase under parser v1; both designs logged and reviewed; the monitor reads the reply receipts
Day 21 (Monday), 10:00	Parser v2 deployed
Day 23 (Wednesday), 09:00	Changepoint alarm at reply receipt 399
Day 24 (Thursday), 09:00	Audit of the 59 replies received since the update; the 15 received after the alarm confirm it
Day 24, 12:00	Rollback to v1; re-parse of the journal since the update
Day 25, 09:00	The monitor restarts with the first receipts after the rollback
Days 28 to 55	Gate window: the receipts the gate reads
Day 56 (Monday)	Promotion gate
After the gate	The next release, with a snooze action and a three-day nag goal, is checked

Table 2: The simulated eight weeks.

3.1 Step 1: the consumers

Table 3 lists what the store is asked. An item is described, as in Part II, by a triple (a, c, e): its age a in days, a flag c that says whether a closing note has been seen, and the time e since that note. The three queries are maps on these triples. The deduplication key is a property of the store rather than of an item: it decides which journal notes belong to the same item.

Consumer	Question	On (a, c, e)	Declared in
Goal engine	is the item open and at most seven days old?	$\neg c \wedge a \leq 7$	`suggest` of Part II's `LiveOpenItems`
Commitment record	is the item open?	$\neg c$	`open` of `LiveOpenItems`
Nag goal	is the item open and at most two days old?	$\neg c \wedge a \leq 2$	`nag` of `WMTMStore`
Deduplication	which notes are about one item?	key (person, topic)	`ItemKey` of `WMTMStore`

Table 3: The consumers of the open-item store.

The list decides the rest of the section. A statistic is sufficient only relative to the queries it must answer, so a store that served the engine alone could forget more than this one may; Part II showed that the coarsest statistic for the engine alone merges closed items with open items past seven days, which the commitment record must keep apart [1, Section 6.1].

3.2 Step 2: the item machine and the abstraction function

The item machine is Part II's `ItemTime`, imported unchanged. Its states are the triples (a, c, e) with $e \leq a$; waiting d days, for any $d \geq 0$, adds d to the age and to the time since the closing note, and a closing note sets c and starts that time at zero, unless the item was closed already. Every claim of this section is about these declarations. Part II has checked its statistics on them, and the example runs those requests again so that its bundle can be checked without Part II's.

The abstraction function connects the machine to the journal. It reads the parsed notes in time order and, for each note, does one of three things. A note parsed as an opening creates the key of its person and topic label, with the note's time as the creation time, if that key does not

exist yet. A note parsed as closing an item applies the action `reply` to the item of its key at the time memory learns of the note: the note's own time when the note is parsed as it is written, and the time of the re-parse for a reply that a re-parse recovers. Later closing notes of the same key change nothing, and notes parsed as anything else are ignored. Between these events, every item waits. A note cannot change a decision made before memory learned of it, which is why the time of learning, and not the note's timestamp, is the time of the action. The abstraction function is code under test, as Part II says [1, Section 8.3], and its errors are the input errors of Section 2.2: a reply the detector parses as an ordinary note, a closing note attached to the wrong item, and a paraphrase that receives a new topic label and therefore a new key.

Time	Person	Topic label	Marker	Parsed	Truth	What memory does
day 10, 18:06	ben	`token-bridge-0288`	open question for Ben	open	open	creates the key; its age counts from here
day 11, 18:52	ben	`token-bridge-0288`	decided	close	spurious	closes the key, though nobody has replied
day 18, 17:52	kofi	`benchmark-run-0500`	pending	open	open	creates the key; its age counts from here
day 22, 15:06	kofi	`benchmark-run-0500`	decided	other	reply	ignored by parser v2; the re-parse at noon on day 24 closes the key
day 50, 17:08	tomas	`demo-script-1235`	open question for Ben	open	open	creates the key; its age counts from here
day 51, 02:40	tomas	`demo-script-1235`	decided	close	reply	first closing note: the key is closed
day 51, 16:46	tomas	`grant-report-1235b`	open question for Ben	open	paraphrase	a second key for the same question

Table 4: The journal notes of three items, from the frozen stream `journal`. The columns up to *Parsed* are what memory sees; *Truth* is kept beside them for the review's labels and the audit's findings. Times are rounded down to the minute.

Table 4 shows the journal notes of three items as the frozen stream holds them. The first, an open question for Ben about `token-bridge-0288`, opened on day 10 and was closed the next evening by a "decided:" note that the detector attached to it in error; nobody had replied. Under the event design the item stayed closed from then on, although it was still open. The second, a pending item concerning Kofi and `benchmark-run-0500`, was answered on day 22 at 15:06, while parser version 2 was deployed, and the parser read the reply as an ordinary note. The item stayed open in memory until the re-parse after the rollback, at noon on day 24, found the reply and closed the item from then on. The third, an item concerning Tomas and `demo-script-1235`, was answered on day 51 at 02:40 and closed correctly. Fourteen hours later a note asked the same question again under the label `grant-report-1235b`, which created a second key for one question; that key was never closed, because the reply belonged to the first. The three items carry the three input errors, and Section 4 follows each of them into the receipts: Kofi's item in Section 4.1, Tomas's second key in Section 4.2 and Ben's item in Section 4.3.

3.3 Step 3: the statistics and the deployed window

The module `WMTMStore` declares the store as deployed. It imports the item machine, the event and joint statistics and the window view from Part II, adds the nag goal and its index, and declares the keyed store.

```
(DC.Module WMTMStore
```

```
  (Profile (Dynamics Deterministic) (Values Crisp) (Carriers Symbolic))
  (Use RationalArith Families Statistics)              ; PROPOSED (Part II)
  ;; the item machine and its statistics, unchanged from Part II
  (Import LiveOpenItems
    (Only Days Item ItemTime EngineTime EventStat JointStat WindowView))

  ;; the nag goal: follow up on an open item while it is at most two days old
  (Map nag (From Item) (To Bool) (Cases ((Tuple $a $c $e) (And (Not $c) (Le $a 2)))))
  (Machine NagTime (States Item) (Observe nag nag) (ActionsOf ItemTime))
  ;; its index: open, past two days, and the age while it is at most two days
  (Map sat-two (From Item) (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $a $c $e)
      (Tuple (Not $c) (And (Not $c) (Lt 2 $a)) (If (And (Not $c) (Le $a 2)) $a 0)))))
  (Map sat-two-wait (Parameters ($d Days)) (From (Tuple Bool Bool Rational))
    (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $o $t $v)
      (Tuple $o (And $o (Or $t (Lt 2 (Add $v $d))))
             (If (And $o (Not (Or $t (Lt 2 (Add $v $d))))) (Add $v $d) 0)))))
  (Map sat-two-close (From (Tuple Bool Bool Rational)) (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $o $t $v) (Tuple False False 0))))
  (Map sat-two-nag (From (Tuple Bool Bool Rational)) (To Bool)
    (Cases ((Tuple $o $t $v) (And $o (Not $t)))))
  (Statistic NagIndex (Of NagTime) (Encode sat-two)
    (Step (wait $d) (sat-two-wait $d) (For $d Days)) (Step reply sat-two-close)
    (Read nag sat-two-nag))

  ;; the store, keyed by the engine's deduplication key: the person and the topic label
  (Carrier Person (Of Symbol) (All))
  (Carrier Topic  (Of Symbol) (All))
  (Carrier ItemKey (Of (Tuple Symbol Symbol)) (Product Person Topic))
  (Store Items (Keys ItemKey) (Of ItemTime) (Global wait))      ; PROPOSED (Part II)
  (Keyed Records (Of Items) (Item EventStat))                  ; durable
  (Keyed EngineIndex (Of Items) (Item JointStat))              ; rebuilt from Records
  (Keyed NagIndexByKey (Of Items) (Item NagIndex)              ; rebuilt from Records
    (View NagTime)))                                           ; PROPOSED View (Part II)

!(dc.run WMTMStore (CheckStatistic EventStat (Method Symbolic)))
!(dc.run WMTMStore (CheckStatistic JointStat (Method Symbolic)))
!(dc.run WMTMStore (CheckStatistic NagIndex (Method Symbolic)))
!(dc.run WMTMStore (CheckNow WindowView))
!(dc.run WMTMStore (CheckUpdates WindowView))
!(dc.run WMTMStore (CheckStatistic Records (Method Lift)))
!(dc.run WMTMStore (CheckStatistic EngineIndex (Method Lift)))
!(dc.run WMTMStore (CheckStatistic NagIndexByKey (Method Lift)))
```

The nag goal is a machine with the actions of `ItemTime` and one query, `nag`. Its index `NagIndex` is the saturation of Part II's Proposition 4.2 at two days [1]: it keeps whether the item is open, whether it is open and past two days old, and the age while it is open and at most two days old. Once an open item passes two days, the index forgets its age, since no consumer of this index asks about it again.

The joint statistic `JointStat` is the index the engine reads. It keeps whether the item is closed, whether it is open and past seven days, and the age of an open item up to seven days, and Part II showed it to be the coarsest statistic that serves the engine and the commitment record together. The event statistic `EventStat` keeps the age and the closed flag with nothing saturated; in the durable record it is two timestamps per key, the creation time and the time from which the item is closed, and the age is computed from them at query time. The window view `WindowView` is the window design: the age, and whether a closing note is visible in a two-day window.

Table 5 lists the verdicts of the eight requests. A `CheckStatistic` request turns a statistic

into obligations, one closure obligation (C) and one update obligation (U) for each action family, and one read obligation (R) for each query [1, Section 3.4]. For the wait family the obligations quantify over every item and every duration, and the solver decides each of them for all values at once. Since the solver's proofs are not checked, a statistic whose obligations are all unsatisfiable is reported as Conditional on cvc5, the solver the reference evaluator calls; Z3 agrees with every such verdict on its own encoding.

Request	Status	Obligations or witness
`CheckStatistic EventStat`	Conditional	six: (C) and (U) for wait and reply, (R) for `suggest` and `open`
`CheckStatistic JointStat`	Conditional	six, as above
`CheckStatistic NagIndex`	Conditional	five: (C) and (U) for wait and reply, (R) for `nag`
`CheckNow WindowView`	Refuted	$x = (3, \text{true}, 3)$ and $y = (3, \text{false}, 0)$: the same view, and only y is to be suggested
`CheckUpdates WindowView`	Refuted	$x = (2, \text{true}, 0)$, $y = (2, \text{true}, 2)$, a wait of 1: the same view before, different views after
`CheckStatistic Records`	Conditional	by lifting `EventStat`; both queries
`CheckStatistic EngineIndex`	Conditional	by lifting `JointStat`; both queries
`CheckStatistic NagIndexByKey`	Conditional	by lifting `NagIndex` to the view `NagTime`; the query `nag`

Table 5: The store as deployed. Conditional: cvc5 found every obligation unsatisfiable and Z3 agrees; no proof is checked. A lifted result carries the assumption of the statistic it lifts.

The two witnesses against the window design are concrete enough to read without notation. The first pair are two items three days old, one answered three days ago and one never answered. Both look open through a two-day window, and the engine proposes a follow-up for both; only the second needs one. The second pair are two items two days old that both look closed today, one answered today and one answered two days ago. A day later the older reply has left the window and its item looks open again, while the other still looks closed, so the view cannot be kept up to date from itself. This is the mechanism behind "stale items stayed open", and a wider window only moves the day on which answered items reappear. Both witnesses were found by the solver and replayed by evaluating the declared maps at the values it returned.

3.4 Step 4: the store, durable records and indexes

The store holds one item per key and applies waiting to every item at once. Three keyed statistics are declared on it, and Part II's lifting proposition [1, Proposition 4.3] carries each item statistic over to the store, with its status and assumptions: the identities hold item by item, and a keyed store of any size is a finite map from keys to items whose operations act on one key, or on all keys alike for the global wait. The proposition has a premise that the lifting request checks before it lifts anything: the item statistic must be a statistic of the store's item machine, `ItemTime`, whose consumers ask both `suggest` and `open`. The nag index is a statistic of `NagTime`, which has the same items and actions and asks only `nag`, so its declaration names that machine as a view with `(View NagTime)`, and the lifted index is bound to the query `nag` alone. Without the view the declaration would be refused as invalid input, since an index certified for one query says nothing about the others.

The protocol asks which statistics are durable and which live only in indexes that can be rebuilt. The example keeps `Records`, the event statistic, as the durable record: two timestamps per key. The engine index and the nag index are compact and fast to read, and both are functions of the record, so they can be dropped and rebuilt at any time. Section 7.5 states the rebuild as a statistic, which the solver establishes, and uses it when the goals change. Figure 1

shows where each part lives.

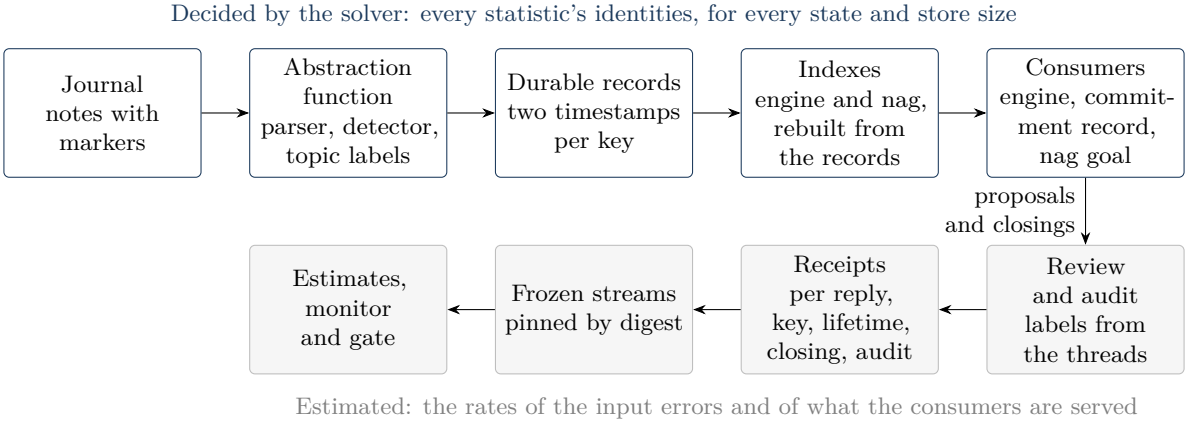


Figure 1: The open-item store. The upper row holds the declared objects, whose claims are checked by a solver; the lower row holds the receipts, from which the rates the declarations cannot reach are estimated.

3.5 Conformance replay

The verdicts of Table 5 are about declarations. Whether the simulated designs behave as the declarations say is a separate question, which Part I answers by replaying recorded events through the code and through the declared machine [2, Section 3.4]. The oracle does this for the whole deployment, starting from the frozen journal. It reads the journal through its own implementation of the abstraction function described above and checks that the result is the simulation's 1,362 keys, with their creation times and the times from which they are closed. It then steps the module's item machine, the event statistic, the joint statistic and the nag index through each key's history, evaluating the module's maps as the reference evaluator reads them. At every daily cycle at which the key was at most seven days old, it compares what the statistics decide, and what the module's window view shows, with the decisions of the two simulated designs. All 8,966 decisions agree, and at every cycle the code of each statistic equals the encoding of the replayed item, which is identity (U) observed on the data.

The replay covers every decision of this corpus, so no sampling bound is needed for it. Both sides of the comparison receive the same events once the journal has been abstracted, so the replay tests the designs' code against the module's maps. It does not test the abstraction function itself: the parser, the detector and the topic labeler reach the claims of this example only through the receipts of the next section.

4 Receipts and streams

Steps 5 and 6 of the protocol record receipts and freeze them. The protocol asked for one receipt per suggestion, carrying the item's status from events, the certificate it cites and the reviewer's label, and for a random sample of items a record of whether the detector found the closing reply [1, Section 9]. The example keeps the random audit as the protocol describes it. It replaces the receipts per suggestion by receipts at other units, for the reason the next subsection gives.

4.1 Why single proposals are the wrong unit

The engine proposes an open item at every cycle until the item is closed or older than seven days. A reply that the detector misses therefore produces a stale proposal at every cycle from the reply to the item's seventh day, so one input error yields a run of labels. Kofi's item of Table 4 was proposed at the cycles of days 19 to 24, and the last two proposals were stale, because the reply of day 22 had been missed. In the gate window the event design logged 1,952 proposals, of which the review labeled 39 stale and 30 duplicate. The 39 stale labels came from 7 keys, two of them stale on seven consecutive days, and the 30 duplicate labels from 5 keys.

Part II estimates rates with a mixture confidence sequence, in the line that begins with Robbins [15] and whose general theory Howard and coauthors give [16]. Read as an estimate of a rate, its interval assumes that every observation has one common conditional rate given the ones before it [1, Section 5.1]. Labels that come in runs from one key break that assumption: once a key has been proposed stale, its next proposal is stale too unless the key ages out, so an interval computed from the 1,952 labels as if they shared one rate would be too narrow. A one-sided claim, that the rate is below a target, needs no common rate: by Part II's Proposition 5.1 it holds for the average, over the observations, of each one's conditional probability given those before it [1, Proposition 5.1]. Even so, the proposals are the wrong unit for the gate's claims, because that average counts a missed reply once for every day its item stays young, and the number of days depends on the log-only policy. Once the engine is live, an answer of "already handled" closes the item at its first stale proposal, so a rate per logged proposal does not carry over to the live engine. The per-proposal labels are kept, as the frozen stream `proposals`, and this text quotes them only as descriptions. For comparison, the window design logged 3,752 proposals in the same four weeks, of which 1,839 were stale.

Kofi's two stale proposals become one observation at the unit used below. His reply produced a single reply receipt, released at the cycle of day 23 and labeled stale, and it was one of the receipts that raised the alarm of Section 5.4.

4.2 Step 5: five units of receipt

Each receipt below has one row per unit, written at a moment fixed in advance, with labels that the unit's own events decide. Table 6 summarizes them.

Stream	One row per	Labels	Rows
`RepliesGate`	reply to an item at most seven days old, at the next cycle	`stale`: the event design proposes the item at that cycle	436
`KeysGate`	key, at its first proposal	`duplicate`: another key holds the same question	453
`LivesGate`	key whose daily decisions all fall in the window, when its seven days end	`window_bad`, `event_bad`: the design erred on the key at some cycle	482
`ClosuresGate`	key, at the first cycle after its first closing	`spurious`: no reply had arrived by the time of the closing note	451
`AuditRepliesGate`	audited item with a reply by age four	`missed`: memory shows it open	140
`AuditOpenGate`	audited item without a reply by age four	`false_close`: memory shows it closed	76

Table 6: The receipts of the gate window, released on days 28 to 55.

A *reply receipt* is written for each reply to an item at most seven days old, at the first cycle

after the reply, and labeled stale when the event design proposes the item at that cycle. For the event design a reply receipt is stale exactly when memory knows of no closing note for the item by the time of the cycle: the detector missed the reply, and no other note closed the item first. The receipt is therefore a measurement, reply by reply, of the detector's misses as the engine experiences them, and its population, the replies, is the same whether or not the engine's proposals reach the prompt. The gate's claims about these receipts are one-sided, so they rest on Part II's Proposition 5.1 and need neither a common rate nor independence between receipts: the claim is about the average, over the replies, of each reply's probability of a stale receipt given the receipts before it. Under a fixed parser that probability is the detector's miss rate, slightly lowered for the replies that a spurious note had closed first. Over the eight weeks 955 replies arrived and 916 reply receipts were written: 31 replies came to items that were past seven days at the next cycle, which the engine no longer proposes, and 8 came after the last cycle, later on day 55. The streams `Replies`, `RepliesBefore` and `RepliesGate` hold the reply receipts of the whole deployment, of the weeks before the update and of the gate window.

The receipts of a cycle are released together, once the review of that cycle is finished, and in the order of the replies' times. A missed reply has a receipt only after the reviewer has found it, so a stream that released each receipt as soon as it existed would put the stale ones last; with the batch rule the order in which a monitor reads the receipts is fixed by the replies' times, whatever their labels.

A *key receipt* is written for each key at its first proposal and carries the review's duplicate label. One paraphrase gives one duplicate key, whatever the number of its proposals. Tomas's second key, `grant-report-1235b`, is an example: the event design proposed it at the cycles of days 52 to 55, and its key receipt, released on day 52, is one row with the label duplicate. The 30 duplicate labels of the gate window come from the five keys that `KeysGate` records as duplicates. A paraphrased key and its original share one question, so their two rows are not independent; the one-sided claim of the gate does not need them to be.

A *lifetime receipt* is written for each key that is not a duplicate and that at least one design proposed, when its seven days end. It carries one label per design, which is 1 when the design erred on the key at some cycle: it proposed the item although the item had been answered, or it did not propose the item while the item was open and the other design proposed it. The second clause records a missed follow-up in the only way the log can show one, through the other design's proposal. The two labels of a row concern the same key at the same cycles, so the rows are pairs, and Part II's comparison of two policies on the same snapshots applies [1, Section 5.4]. That comparison is a sequential form of McNemar's test for paired proportions [17]: only the keys on which exactly one design erred carry information about which design errs more, and the claim that the window design errs more often is one-sided, so Proposition 5.1 of Part II covers it as well. The stream `LivesGate` keeps the receipts of the keys whose daily decisions all fall in the gate window, which are the keys created after the cycle of day 27 whose seven days ended by the cycle of day 55. A key created earlier would bring decisions from before the window into it, including some from the days of the regression. An error that both designs make at the same cycle, such as withholding an item that both see as closed, leaves no trace in these labels; the next unit is the one that sees it.

A *closure receipt* is written for each key that is not a duplicate at the first cycle after memory learned of its first closing, and carries the review's label: spurious when no reply had arrived by the time of the closing note. Under the event design the first closing decides the item for good, so the closure receipts measure, closing by closing, the false closings that the engine inherits.

An *audit receipt* is written for each item in the random sample when it reaches the age of four days. The sample is drawn when the item is created, a third of all items. For an item that had a reply by then, the label says whether memory still shows it open; for an item without a reply, whether memory shows it closed. The two kinds go to two streams, because they estimate

two different rates. Both describe memory at the age of four days. An error that arises later is outside them: with the simulation's delays, about one spurious closing note in fourteen arrives after the fourth day.

4.3 Where the labels come from

In a live deployment the labels have different sources, and each source has something it cannot see. A reply the detector found is known to memory. A reply it missed becomes known when the reviewer labels the item's next proposal stale and finds the reply in the thread; the reply receipt is then written with the reply's time and the label 1. A missed reply that the reviewer also overlooks produces no receipt at all, so the reply streams undercount misses at the rate at which the review overlooks them. The random audit is the independent check, since its sample is fixed before anyone knows how the item will go and the auditor reads the whole thread.

False closings are invisible to the review of proposals, because an item closed by a spurious note is never proposed by the event design again. The review of closings sees them, since it reads the thread of every first closing. Ben's question about `token-bridge-0288` is the example. The event design proposed it at the cycle of day 11, before the spurious note arrived that evening, and never again. The review of closings labeled the note spurious at the cycle of day 12; in the log-only phase nothing feeds a label back to memory, so the item stayed closed under the event design. The window design proposed it again on days 14 to 17, once the note's two days in the window had passed, and for this item that was the right decision. The item was also in the random sample, and its audit on day 14, at the age of four days, recorded it as open in the thread and closed in memory. A false closing is the one failure that the event design makes worse than the window design. The window design stops counting a closing note after two days, so a false closing costs it at most two days of proposals, while the event design keeps the closing until someone corrects it.

Over the eight weeks the random audit found one false closing among 159 audited items without a reply, and six missed replies among 279 audited items with one. The review of closings labeled three first closings spurious: Ben's, and two in the gate window.

4.4 Step 6: freezing the receipts

The simulation writes every receipt to a frozen stream, a JSON file with a schema, a provenance record and the rows, written once in a fixed byte layout so that its SHA-256 digest is reproducible. The module `WMTMReceipts` declares the ten streams that its requests read, each with its source and digest, and with the three fields that say what an estimate from it is about: the population it samples, the rule that selects its rows, and the policy under which the rows were produced. In a live system the policy field holds the digest of the deployed code and flags; here it names the phase and the parser version. An estimate does not carry over to another policy without an argument; Section 7.6 gives the arguments for the next release and shows how a small change to its design would defeat one of them. The reference evaluator refuses a stream whose file does not match its declared digest, before it counts anything: the oracle sets every stale label of `RepliesGate` to zero in a copy of the bundle, and the request on it returns InvalidInput with no count and no interval. The listing below holds the streams and the monitors, which Section 5 explains. Two further streams, the parsed journal and the proposal log, are frozen for the conformance replay and for the descriptions above; Appendix C lists all twelve with their digests.

```
(DC.Module WMTMReceipts
  (Profile Stochastic)
  (Use Decisions Laws Families Monitors Estimates)     ; PROPOSED Estimates (Part II)
```

```
  ;; one receipt per reply, released at the next cycle: was the item proposed (stale)?
  (Stream Replies (Source "streams/replies.json")
    (Digest "sha256:1f5d10eff762371dffdef5e0be4f940e39366a8fe2c6b9227bb7cdb66c82ca0f")
    (Provenance "simulated: replies to items at most seven days old, days 0-55")
    (Population "replies to items at most seven days old at the next cycle")
    (Selection "one row per reply at the next cycle, after its review, by reply time")
    (Policy "log-only phase; parser v1, v2 from day 21 10:00 to day 24 12:00"))
  (Stream RepliesBefore (Source "streams/replies_before.json")
    (Digest "sha256:3a5629468d68aba06e30587dfc23114cbf4ed6e545e426ed4d77c08caf799703")
    (Provenance "simulated: the reply receipts of days 0-21, before the parser update")
    (Population "replies to items at most seven days old at the next cycle")
    (Selection "the rows of Replies released on days 0-21")
    (Policy "log-only phase; parser v1"))
  (Stream RepliesGate (Source "streams/replies_gate.json")
    (Digest "sha256:39def219d4692915bb2d0ce2680d8a85b5aee69729e5f5c419c5c8beb01a764e")
    (Provenance "simulated: the reply receipts of days 28-55")
    (Population "replies to items at most seven days old at the next cycle")
    (Selection "the rows of Replies released on days 28-55")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
  ;; one receipt per key at its first proposal: the review's duplicate label
  (Stream KeysGate (Source "streams/keys_gate.json")
    (Digest "sha256:fd12fdce6989efac157bd53aff04b5e71b40dc710733fedfba64e8caed1ee5e8")
    (Provenance "simulated: keys first proposed on days 28-55")
    (Population "keys that the event design proposes")
    (Selection "one row per key at its first proposal, days 28-55")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
  ;; one receipt per key at the end of its seven days: did each design decide wrongly?
  (Stream LivesGate (Source "streams/lives_gate.json")
    (Digest "sha256:5eb1f54bf60f047de30f1cdfa4516dcefc3a789df5bac4763cdebe14aaaca801")
    (Provenance "simulated: keys whose daily decisions all fall on days 28-55")
    (Population "keys that are not duplicates and that either design proposed")
    (Selection "one row per key whose decisions all fall on days 28-55")
    (Policy "log-only phase; both designs; parser v1"))
  ;; one receipt per key at its first closing: was the closing note a real reply?
  (Stream ClosuresGate (Source "streams/closures_gate.json")
    (Digest "sha256:b11d52683007c2f3f2c25e367ee66f81e4e3393360e3ae9d297fd699806ff209")
    (Provenance "simulated: first closings learned between the cycles of days 27 and 55")
    (Population "first closings of keys that are not duplicates")
    (Selection "one row per key at the cycle after memory learns its first closing")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
  ;; the random audit at age four days, and the targeted audit after the alarm
  (Stream AuditRepliesGate (Source "streams/audit_replies_gate.json")
    (Digest "sha256:8f9a06bed6d5b60ee2007dde3cafd41ff198a57cdc572d7baa59251bf63b97ec")
    (Provenance "simulated: audited items with a reply by age four, days 28-55")
    (Population "items, sampled at creation with probability 1/3")
    (Selection "one row per sampled item replied by age four, audited at four")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
  (Stream AuditOpenGate (Source "streams/audit_open_gate.json")
    (Digest "sha256:840aff688d32968a5d7d2d9c15fc132ca416d109418f260686803037267e8db5")
    (Provenance "simulated: audited items without a reply by age four, days 28-55")
    (Population "items, sampled at creation with probability 1/3")
    (Selection "one row per sampled item not replied by age four, audited at four")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
  (Stream AuditAfterAlarm (Source "streams/audit_after_alarm.json")
    (Digest "sha256:5d6f7667bbdf76e094d5827d64beb8b36b1c5f959b40af405d1844c743d6b0fc")
    (Provenance "simulated: audited replies received after the receipts of the alarm")
    (Population "replies received from the alarm cycle to the audit on day 24")
    (Selection "every such reply, audited on day 24 at 09:00")
    (Policy "log-only phase; parser v2"))
  (Stream AuditV2 (Source "streams/audit_replies_v2.json")
    (Digest "sha256:ea80e318024f8534248b4d58a075b9243dfc1ad6dfa71d842a7bd74bfbef0038")
    (Provenance "simulated: every reply between the parser update and the audit")
    (Population "replies received from the parser update to the audit on day 24")
```

```
    (Selection "every such reply, audited on day 24 at 09:00")
    (Policy "log-only phase; parser v2"))

  ;; the monitors: a missed reply that reaches a proposal, accepted at one in twenty
  (Carrier One  (Of Symbol) (Enum one))
  (Carrier Rate (Of Symbol) (Enum accepted tenth fifth twofifths))
  (Map miss-rate (From Rate) (To Rational01)
    (Table (accepted (Rat 1 20)) (tenth (Rat 1 10)) (fifth (Rat 1 5))
           (twofifths (Rat 2 5))))
  (Map stale (Parameters ($r Rate)) (From One) (To (Law Nat))
    (Body ($u) (Coin (miss-rate $r))))
  (Monitor MissTenth (Context One) (Null (stale accepted)) (Alternative (stale tenth))
    (Columns (observation "stale")))
  (Monitor MissFifth (Context One) (Null (stale accepted)) (Alternative (stale fifth))
    (Columns (observation "stale")))
  (Monitor MissTwoFifths (Context One) (Null (stale accepted))
    (Alternative (stale twofifths)) (Columns (observation "stale"))))
```

The gate reads the receipts released on days 28 to 55, a window fixed before it began, and the regression of the fourth week lies outside every stream it reads. The reply, key and closure receipts of the window concern replies, proposals and closings after the cycle of day 27. The audits of the window concern items created from day 24 on, whose replies were all parsed by version 1, at the latest by the re-parse at noon on day 24. The lifetime receipts are restricted, as described above, to keys created after the cycle of day 27.

5 The regression

Step 8 of the protocol runs a mixture monitor on the stale rate. In the calendar it comes first, since the monitor reads the reply receipts from day 0, and it is the step that met the one incident of the eight weeks.

5.1 Step 8: the monitors

The team accepts a stale reply receipt rate of one in twenty. Parts I and II stated that rate per suggestion; the example states it per reply, the unit at which it can be estimated (Section 4.1). Once the engine is live and an answer of “already handled” closes the item, each stale reply receipt costs one stale proposal, and in the gate window there were about four proposals for every reply receipt, so one in twenty per reply is the stricter of the two targets. The module declares three monitors, each comparing the accepted rate with one failing rate, 1/10, 1/5 or 2/5, as Part II’s Pattern 8 did [1, Section 6.8]. For a receipt with label X_n and an alternative rate q against the accepted rate $p_0 = 1/20$, a monitor’s factor is the likelihood ratio

$$L_n = \left(\frac{q}{p_0}\right)^{X_n} \left(\frac{1-q}{1-p_0}\right)^{1-X_n},$$

which exceeds one for a stale receipt and falls short of one otherwise. The requests:

```
!(dc.run WMTMReceipts (CheckMonitor MissTenth))
!(dc.run WMTMReceipts (CheckMonitor MissFifth))
!(dc.run WMTMReceipts (CheckMonitor MissTwoFifths))
!(dc.run WMTMReceipts
   (Detect MissTenth MissFifth MissTwoFifths (On Replies) (Level (Rat 1 100))
           (Combine Mixture)))
!(dc.run WMTMReceipts
   (Detect MissTenth MissFifth MissTwoFifths (On Replies) (Combine Mixture)
           (Scheme ShiryaevRoberts) (MeanTimeToFalseAlarm 100000)))   ; PROPOSED Scheme
```

```
!(dc.run WMTMReceipts
   (EstimateRate (On RepliesBefore) (Column "stale") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On AuditAfterAlarm) (Column "missed") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On AuditV2) (Column "missed") (Equals 1) (Level (Rat 1 100))))
```

A monitor declares the context in which each observation is made, and these three have a single context, the value `one` of the carrier `One`. `CheckMonitor` is therefore one identity for each monitor, and it is Checked for all three by exact arithmetic, without a solver: under the accepted rate the factor's expectation is exactly one. The two `Detect` requests combine the three monitors as a mixture and differ in how they accumulate evidence. The first is Part II's scheme, which the patch calls `Ville`: each monitor multiplies its factors from the first receipt on, the mixture averages the three products, and the alarm is raised when the average reaches $1/\alpha = 100$. Ville's inequality bounds the probability of ever raising a false alarm by $\alpha = 0.01$, however long the monitor runs [13, 14]. This α is the *level* of the guarantee, the probability of error it allows, and every estimate in this example is stated at the same level: an interval at level 0.01 fails to contain the true rate with probability at most 0.01. The second `Detect` request is the changepoint scheme of Section 5.4, which the deployment ran in the loop. The remaining requests are the estimates of Section 5.5.

5.2 What happened

Parser version 2 went live on day 21, a Monday, at 10:00. It changed how the reply detector matched a "decided:" note to an open item, and in the simulation it misses a reply with probability 1/4, against 0.015 for version 1. Nothing in the declarations changed. The detector is part of the abstraction function, outside every declaration, and the statistics went on answering correctly for the events they were given, so the change showed only in the receipts. Receipt 362 was the last for a reply received before the update; of the 37 receipts from receipt 363 to receipt 399, released at the cycles of days 22 and 23, ten were stale.

5.3 Why a monitor started at the first receipt stays silent

Part II's monitor never fired. Its value after the first receipt, 0.807, was the largest it reached over the whole stream. By receipt 362 the logarithm of its value had fallen to -16.93; the alarm needs the logarithm to reach $\ln 100 = 4.61$; and by receipt 399, after the ten stale receipts, it stood at -11.46. After the rollback it fell again, to -31.2 at the last of the 916 receipts.

Each receipt that is not stale multiplies every monitor's product by $(1 - q)/(1 - p_0) < 1$, so three healthy weeks leave every product, and hence their average, far below one. A change that comes after them must first make up that factor before the monitor can count any evidence against the accepted rate. The monitor answers the question it was built for, whether the stream since the first receipt is consistent with the accepted rate, and after 362 healthy receipts a few dozen bad ones take a long time to change that answer. Part II's simulations of monitors used streams whose rate was the same from the first receipt on [1, Section 5.5], where this does not arise.

A counterfactual run of the same simulation without the rollback shows the cost. With version 2 left in place, Part II's monitor would have fired at receipt 524 on day 30, a week after the changepoint monitor. On days 24 to 30 the event design would have logged 169 stale proposals, where the run with the rollback logged 21: the late alarm would have cost 148 stale proposals.

5.4 A changepoint monitor

The statistics of quickest detection address this case: observations follow one law until an unknown time and another afterwards, and an alarm should come soon after the change and rarely before it. Page's cumulative sum [7] and the procedure of Shiryaev and Roberts [8, 9] are the classical answers, with optimality results by Lorden [10] and Pollak [11]. Shin, Ramdas and Rinaldo built such procedures from e-processes, nonnegative processes whose expected value under the null hypothesis is at most one at every stopping time [12]. That construction extends them to composite null hypotheses, which allow a range of rates, such as "the rate is at most p_0", instead of a single one. The example uses the Shiryaev–Roberts form. For each alternative q, let $R_0 = 0$ and

$$R_n = (1 + R_{n-1})\, L_n, \qquad \text{equivalently} \qquad R_n = \sum_{k=1}^{n} \prod_{j=k}^{n} L_j.$$

The term for k is the likelihood ratio of "the rate became q at receipt k" against "no change", over the receipts from k to n, so R_n adds the evidence for every possible start of a change. The term for $k = 1$ is the product of Part II's monitor. The mixture $\bar{R}_n$ averages R_n over the three alternatives, and the alarm is raised at the first n with $\bar{R}_n \geq A$.

Proposition 5.1 (False alarms of the changepoint monitor)**.** *Let $X_1, X_2, \ldots$ be binary labels such that, for every n, the conditional probability that $X_n = 1$ given $X_1, \ldots, X_{n-1}$ is at most p_0. Let $q_1, \ldots, q_m > p_0$, let $R_n^{(i)}$ be the statistic above for q_i, $\bar{R}_n$ their average, and T the first n with $\bar{R}_n \geq A$, for some $A > 0$. Then*

1. *$\mathbb{E}[T] \geq A$, and*
2. *$P(T \leq N) \leq N/A$ for every $N \geq 1$.*

Proof. Write p_n for the conditional probability that $X_n = 1$ given the past. The conditional expectation of the factor for q_i is $p_n q_i/p_0 + (1 - p_n)(1 - q_i)/(1 - p_0)$, which increases with p_n because $q_i > p_0$ and equals one at $p_n = p_0$, so it is at most one. Hence the conditional expectation of $R_n^{(i)}$ is at most $1 + R_{n-1}^{(i)}$, and the same holds for the average: the conditional expectation of $\bar{R}_n$ is at most $1 + \bar{R}_{n-1}$.

For the first part, $\bar{R}_n - n$ is then a supermartingale that starts at zero: its conditional expectation, given the past, never exceeds its last value. For every k the stopping time $T \wedge k$ is bounded, and optional stopping gives $\mathbb{E}[\bar{R}_{T\wedge k}] \leq \mathbb{E}[T \wedge k]$. Since $\bar{R}$ is nonnegative and at least A at time T, the left side is at least $A\, P(T \leq k)$. Letting k grow gives $\mathbb{E}[T] \geq A\, P(T < \infty)$. If $P(T < \infty) = 1$ this is the claim, and otherwise $\mathbb{E}[T]$ is infinite; hence $\mathbb{E}[T] \geq A$.

For the second part, fix N and let $M_n = \bar{R}_n + (N - n)$ for $n = 0, \ldots, N$. Then $M_0 = N$, every M_n is nonnegative, and the conditional expectation of M_n is at most $1 + \bar{R}_{n-1} + N - n = M_{n-1}$, so M is a nonnegative supermartingale. Ville's inequality bounds the probability that it ever reaches A by $M_0/A = N/A$. On the event $T \leq N$, $M_T \geq \bar{R}_T \geq A$. Hence $P(T \leq N) \leq N/A$. □

In plain terms. A monitor that adds up the evidence for a change starting at every possible receipt will raise a false alarm sooner or later, but not soon. With the threshold at 100,000, it reads at least 100,000 receipts on average before its first false alarm when nothing has changed, and the chance of a false alarm within any N receipts is at most N in 100,000.

The guarantee is of a different kind from Ville's. Ville's inequality bounds the probability of any false alarm over an unbounded future. The proposition bounds the expected number of receipts before a false alarm, and its probability over any fixed number of receipts, but not over an unbounded future: if each receipt is stale with some positive probability, independently of

the others, the stream contains, with probability one, a run of stale receipts long enough to carry $\bar{R}_n$ past any threshold. The ability to detect a late change is paid for with false alarms at long intervals, at least A receipts apart on average if the monitor restarts after each. An alarm is then a reason to look, and the look has to produce evidence that the alarm did not use, which is what the confirmation rule of Section 5.5 requires. The deployment set $A = 100{,}000$. At 16.4 receipts a day that is at least 16.7 years of receipts, on average, before a false alarm. By the second part, the probability of a false alarm was at most 0.0092 over the 916 receipts of the eight weeks, and it is at most 0.06 over a year of receipts at the same rate. The detection delay of such procedures grows only with the logarithm of A [10, 11]. Declaring this monitor needs two new arguments of `Detect`, the scheme and the mean time to a false alarm, which the registry patch of Appendix B adds.

At the cycle of day 23 the monitor read the receipts released that morning, and at receipt 399 the mixture reached 116,558, whose logarithm 11.67 exceeds $\ln 100{,}000 = 11.51$. The alarm came 47 hours after the update; at the update the logarithm of the mixture had been 2.12. The monitor stopped at its alarm, so receipts 400 to 419, the rest of that morning's batch and the batch of day 24, were read by no monitor. Figure 2 shows the two monitors on the whole stream.

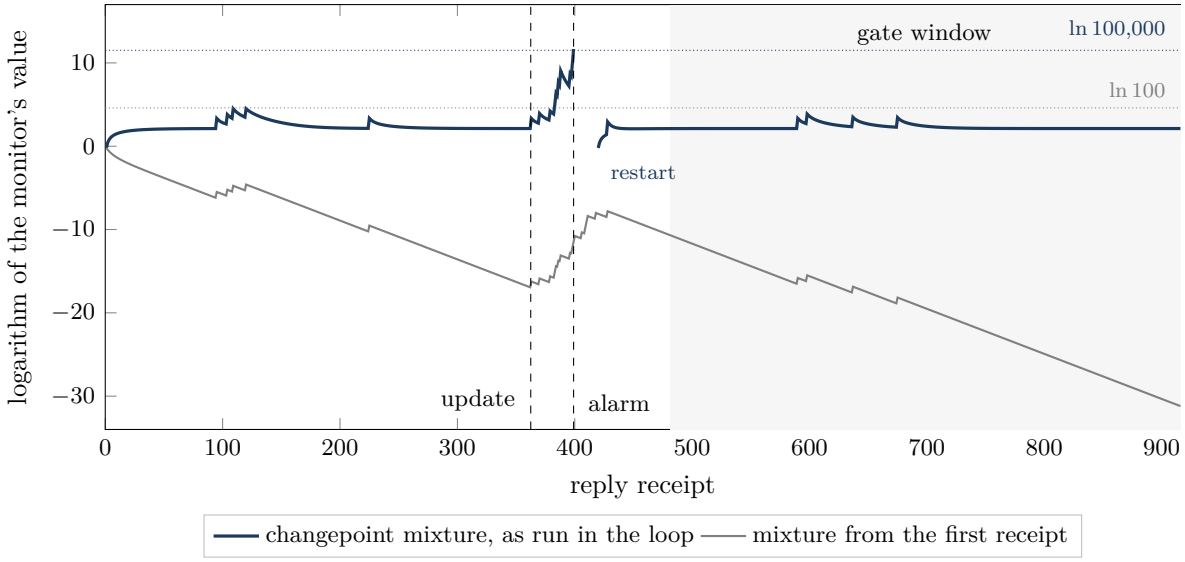


Figure 2: The two monitors on the frozen stream `Replies`. The changepoint mixture is shown as it ran in the loop: it crosses its threshold at receipt 399, reads no receipt from 400 to 419, and restarts at receipt 420, the first of the cycle of day 25. The `Detect` request, which reads the stream from the first receipt, reports its first alarm at the same receipt. The mixture from the first receipt never reaches its own threshold. The dotted lines are the two thresholds, and the shaded region holds the receipts of the gate window.

5.5 Confirming and repairing

The alarm says that the evidence for a change has passed the threshold. It does not say how large the change is, and its guarantee allows false alarms at long intervals, so the team had fixed its response before the deployment began. On the morning after an alarm, an auditor reads the threads of the replies received since the last parser update. The team rolls the parser back only if the replies received after the alarm's receipts, which the alarm did not use, give a confidence sequence for the miss rate whose lower end lies above the accepted 1/20. That is a one-sided claim against a fixed target, so by Part II's Proposition 5.1 it holds at level 0.01 for

the average miss rate of those replies, whether or not they share one rate.

On day 24 at 09:00 the auditor read the threads of all 59 replies received since the update and found 17 that memory did not show. Of the 59, 44 had been received before the alarm's cycle, and 37 of these were the receipts from 363 to 399 on which the alarm rested. The other 15 arrived after the alarm, between the cycles of days 23 and 24, and memory had missed 5 of them. Their interval at level 0.01, [0.0512, 0.7525], lies above 1/20, so data that the alarm had not seen confirmed it. Table 7 lists the estimates and the monitors. The audit of all 59 replies puts the miss rate since the update between 0.1144 and 0.5202, and the reply receipts of the three weeks before the update put the stale rate between 0.0009 and 0.0565. The two intervals do not overlap, and each contains the rate the simulation used.

Request	Status	Result
`CheckMonitor` for each of the three	Checked	the factor's expectation under 1/20 is one
`Detect`, scheme `Ville`	Estimated	no alarm in 916 receipts; largest value 0.807, at receipt 1
`Detect`, scheme `ShiryaevRoberts`	Estimated	first alarm at receipt 399, mixture 116,558
`EstimateRate` on `RepliesBefore`	Estimated	5 stale of 362; [0.0009, 0.0565]
`EstimateRate` on `AuditAfterAlarm`	Estimated	5 missed of 15; [0.0512, 0.7525]
`EstimateRate` on `AuditV2`	Estimated	17 missed of 59; [0.1144, 0.5202]

Table 7: The monitors and the incident's estimates, at level 0.01.

At noon the parser was rolled back to version 1, and the journal written since the update was parsed again. Of the 64 replies received while version 2 was deployed, it had missed 18; the audit had covered 59 of the 64, and the other five came in the three hours between the audit and the rollback. The re-parse recovered all 18, and each closed its key from noon on day 24, when memory learned of it. Before that, the missed replies had produced 28 stale proposals in the log, on days 22 to 24. The monitor restarted at the cycle of day 25, with receipt 420. Over the 497 receipts that followed, its largest value was 47.63, far below the threshold, and there was no other alarm in the eight weeks.

The incident left the declarations and their certificates as they were. Part II states the reason among its limits: a statistic is exactly as good as the events it is updated with [1, Section 10]. What the example adds is a procedure around that limit, in which the monitor on the receipts raised the alarm and an audit of replies the alarm had not used confirmed it before the re-parse corrected the durable records. Every index can then be rebuilt from the corrected records, since each index is a function of them (Section 7.5).

6 The promotion gate

Step 7 of the protocol estimates the stale-suggestion rate, the detector's miss rate and the paired difference between the two designs, and the gate on day 56 reads the estimates together with the certificates of Section 3.

6.1 What the gate decides

One candidate arrives at the gate: `goal_prompt` turns on, and the engine decides from the store's engine index, the joint statistic lifted to the store, instead of from its recall window. The candidate changes what the engine reads and nothing else, so it touches one consumer, the engine's queries. Part I declared that consumer with the agent's custody: the agent may propose changes to it, and each change is a candidate that passes the gate. Consumers with the

frame's custody, such as the commitment record, are held outside the agent's authority and change only through an external ceremony [2, Section 3.5]. GOLEM-Iter sorts candidates into authority classes by the most authority they could plausibly need, and a change to a persistent memory policy, such as this one, is class A2 [2, Section 2.3].

The candidate arrives with a certificate bundle: the `CheckStatistic` results for the touched consumer, and the Estimated rates on which its empirical claims rest. Every result in the bundle is bound to the machine and queries it was checked against, and the gate compares that binding, the result's status and its assumptions with the consumer's declaration. The symbolic results are Conditional on cvc5. The engine's queries are an operational consumer with the agent's custody, and their declaration accepts Checked results and results whose only assumption is the cvc5 backend, `(Accepts Checked (Conditional (TrustedBackend cvc5)))`; a consumer with the frame's custody would need the solver's proofs checked first [2, Section 3.5], [1, Section 7.2]. The gate has a lifetime confidence budget, the total probability of error it allows itself over all its decisions. Certificates spend nothing from it, whether Checked or Conditional on a solver, and each Estimated result spends its level [2, Section 12.3], [1, Section 8.3]. Part I's gate rule is that a Refuted result for a touched consumer blocks promotion and that Estimated results with too little data behind them are a reason to keep the candidate in shadow, where it runs beside the deployed version without affecting what users see. The claims, their targets and the gate window were fixed before day 28.

6.2 Step 7: the claims

The certificates are those of Table 5: `EventStat`, `JointStat` and `NagIndex` are Conditional on cvc5, the three keyed statistics follow by lifting with the same assumption, the window design that the candidate replaces is Refuted twice, and the conformance replay of Section 3.5 passed on all 8,966 decisions. The Estimated claims are the six requests below, each at level 0.01. Four of them are the candidate's claims and rest on the review's labels. The other two come from the random audit and check the review.

```
!(dc.run WMTMReceipts
   (EstimateRate (On RepliesGate) (Column "stale") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On KeysGate) (Column "duplicate") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On ClosuresGate) (Column "spurious") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (CompareRates (On LivesGate) (First "window_bad") (Second "event_bad") (Equals 1)
                 (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On AuditRepliesGate) (Column "missed") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On AuditOpenGate) (Column "false_close") (Equals 1)
                 (Level (Rat 1 100))))
```

6.3 Reading the results

Table 8 lists the results. Four of 436 reply receipts were stale, and the confidence sequence puts the rate between 0.0004 and 0.0436, below the target. Five of the 453 keys first proposed in the gate window were duplicates, and the interval ends at 0.0460. Two of the 451 first closings were spurious, and the interval ends at 0.0339. For E3, of the 482 keys whose daily decisions all fall in the gate window, the window design erred on 314 on which the event design did not, the event design on none on which the window design did not, and both on 4. The comparison uses only the 314 discordant keys, and its interval for the window design's share of them, $[0.9676, 1]$, lies

	Claim	Count	Interval at 0.01	Established
The candidate's claims, from the review				
E1	fewer than one reply in twenty is followed by a stale proposal	4 of 436	[0.0004, 0.0436]	yes
E2	fewer than one key in twenty is a duplicate	5 of 453	[0.0007, 0.0460]	yes
E3	the window design errs on more keys than the event design	314 against 0	[0.9676, 1]	yes
E6	fewer than one first closing in twenty is spurious	2 of 451	[0.0000, 0.0339]	yes
The audit's check of the review, at the age of four days				
E4	memory misses fewer than one audited reply in twenty	2 of 140	[0.0001, 0.0969]	not yet
E5	memory closes fewer than one audited open item in twenty	0 of 76	[0, 0.1111]	not yet

Table 8: The Estimated claims of the gate, on the receipts of days 28 to 55. A claim with the target one in twenty is established when the upper end of its interval is below 1/20; E3 is established when its interval, for the window design's share of the discordant keys, lies above one half. The lower end of E6 is 0.000015.

far above one half. The intervals of E1, E2 and E4 to E6 each contain the value the simulation used: 0.015 for the miss rate, 0.0087 for the share of duplicates among proposed keys, 0.0148 for the share of audited replies that memory misses, 0.0112 for false closings by the age of four days, and 0.0103 for spurious first closings.

One of the two spurious closings shows why E6 is needed beside E3. A spurious note closed the item `i0829` late on day 34, and its real reply came on the afternoon of day 35. At the cycle of day 35 the item was open and neither design proposed it, since both had seen the closing note. The lifetime receipt has no label for an error that both designs make at the same cycle, and it records the window design's later stale proposals of days 37 to 39 instead; the closure receipt is the only receipt that sees the event design's error.

E4 and E5 do not reach their target, and neither result is evidence of a problem. Two misses in 140 and no false closing in 76 are far below one in twenty; the audit is too small to say so at level 0.01. For E5 the interval is the zero-failure bound of Part II, $1 - (\alpha/(n+1))^{1/n}$ with $n = 76$ [1, Section 5.3]. Both describe memory at the age of four days, so they are lower bounds for the error over an item's whole life (Section 4.2).

6.4 The decision

The candidate is promoted on conditions. `goal_prompt` goes on, and the engine decides from the store's engine index. The certificates cover the touched consumer with a status it accepts, nothing is Refuted, and E1, E2, E3 and E6 are established.

The candidate's absolute error budget is one in twenty for each kind of error the engine can make: a stale proposal after a reply, which E1 bounds; a duplicate key, which E2 bounds; and a follow-up withheld from an open item, which E6 bounds. Under the event design an open item at most seven days old goes unproposed only when a closing note has closed it, and a closing note on an open item is a spurious closing, so E6 is the bound on withheld follow-ups. E3 is a comparison. It shows that the event design errs on fewer keys than the window design, which bounds no absolute harm: a design can beat a poor comparator and still miss too many follow-ups. The decision therefore rests on E1, E2 and E6 for the budget, and on E3 only for the choice between the two designs.

The four established claims rest on one hypothesis that their receipts cannot test: that the

review finds the errors it labels, the missed reply behind a stale proposal, the paraphrase behind a duplicate key and the spurious note behind a false closing. The bundle records the hypothesis with the claims, as Part II asks receipts to record the hypotheses of the guarantees they cite [1, Section 8.3], and names E4 and E5 as its check. After four weeks the check is too small to establish its targets, which by Part I's rule is a reason to shadow. The gate weighed that reason against what the audit did find. Both misses it found in the gate window had stale reply receipts, so the review had found them too, and it found no false closing. In the simulation the review makes no mistakes, and these numbers could not have come out otherwise; in a live deployment they are the first evidence about the review, and the reason to keep collecting it.

The conditions are three. The random audit continues at its current rate, the window design keeps running in shadow so that lifetime receipts, and with them E3, continue to be written, and the candidate returns to shadow if the interval of E4 or E5 comes to lie above one in twenty. A team that read Part I's rule strictly would keep the candidate in shadow until E4 and E5 are established, which the planning below puts months away at the current audit rate.

Had the engine been live in the gate window, with an answer of "already handled" closing the item, each of the seven keys that the event design proposed stale would have been shown stale once: 7 stale proposals among 1,920. That figure counts proposals and is a description, for the reasons of Section 4.1; E1 is the claim.

6.5 Planning the audit

Part II's planning table gives the number of observations at which the upper end of the interval falls below a target when the observed rate equals the true rate [1, Section 5.6]. That is a plug-in size. At the rates the simulation used it is 521 audited replies for E4 and 414 audited open items for E5, and at those sizes the claims would be established only 63 and 68 times in a hundred, because the observed count varies around its mean. A binomial calculation gives the sizes from which on each claim is established with probability at least 0.9: 776 audited replies and 587 audited open items. The gate window yielded 35 audited replies and 19 audited open items a week, so the audit streams reach those sizes about 22 and 31 weeks after the start of the gate window. Auditing every item at the age of four days, instead of a third, would cut both times to a third. A team planning this in a live deployment would use a pessimistic rate instead of the simulation's, and the times would be longer. For comparison, bounding the reply receipts' rate below 3/100 instead of 1/20 with the same probability would take 3,304 receipts, about 30 weeks at the gate window's rate.

6.6 The gate's record

The candidate goes to the gate with a bundle in the format of Part I [2, Appendix C.6], with one field added for the hypothesis and one for the conditions. Entries are cited by the digests of the certificate store; the stream digests are those of Appendix C, shortened here, and the conformance corpus is the frozen journal.

```
{"candidate": "sha256:...", "authority_class": "A2",
 "touched": [{"consumer": "EngineQueries", "custody": "Agent",
              "accepts": "Checked (Conditional (TrustedBackend cvc5))"}],
 "results": [{"request": "(CheckStatistic EventStat (Method Symbolic))",
              "status": "Conditional", "entry": "sha256:...",
              "bound_to": ["ItemTime", ["open", "suggest"]],
              "assumptions": ["(TrustedBackend cvc5 1.4.2 ALL sha256:...)"],
              "agreed_by": ["z3 5.3.3"]},
             {"request": "(CheckStatistic JointStat (Method Symbolic))",
              "status": "Conditional", "entry": "sha256:...", "...": "as above"},
```

```
            {"request": "(CheckStatistic Records (Method Lift))",
             "status": "Conditional", "entry": "sha256:...", "...": "as above"},
            {"request": "(CheckStatistic EngineIndex (Method Lift))",
             "status": "Conditional", "entry": "sha256:...", "...": "as above"}],
"digests": {"interface": "sha256:...", "abstraction": "sha256:...",
            "code": "sha256:..."},
"estimated": [{"claim": "E1", "stream": "sha256:39def219...", "count": 4,
               "observations": 436, "interval": [0.000367, 0.043641]},
              {"claim": "E2", "stream": "sha256:fd12fdce...", "count": 5,
               "observations": 453, "interval": [0.000722, 0.046001]},
              {"claim": "E3", "stream": "sha256:5eb1f54b...", "first_only": 314,
               "second_only": 0, "interval": [0.967552, 1.0]},
              {"claim": "E6", "stream": "sha256:b11d5268...", "count": 2,
               "observations": 451, "interval": [0.000015, 0.033882]},
              {"claim": "E4", "stream": "sha256:8f9a06be...", "count": 2,
               "observations": 140, "interval": [0.000086, 0.096927]},
              {"claim": "E5", "stream": "sha256:840aff68...", "count": 0,
               "observations": 76, "interval": [0.0, 0.111081]}],
"level": "1/100",
"policy": "log-only phase; parser v1, journal re-parsed on day 24",
"budget": {"E1": "1/20", "E2": "1/20", "E6": "1/20"},
"hypotheses": [{"claims": ["E1", "E2", "E3", "E6"],
                "statement": "the review finds the errors it labels",
                "checked_by": ["E4", "E5"]}],
"conformance": {"corpus": "sha256:afe7936b...", "decisions": 8966,
                "result": "passed"},
"decision": "promote",
"conditions": ["random audit continues at one item in three",
               "window design stays in shadow; lifetime receipts continue",
               "return to shadow if the lower end of E4 or E5 exceeds 1/20"]}
```


6.7 The confidence budget

The six Estimated claims spend $6 \times 0.01 = 0.06$ of the gate's lifetime budget; the certificates and the conformance replay spend nothing. The three estimates of the incident in Section 5.5 are not gate claims, and a union bound over all nine estimates of the example bounds by 0.09 the probability that any of their intervals fails to contain its true rate.

Reading E4 and E5 again at a later gate costs nothing more. Each interval is a confidence sequence, valid at every row of its stream simultaneously, so if the audit streams are extended rather than restarted and the claims stay the same, the later reading is covered by the 0.01 already spent. This is the practical reason for anytime-valid estimates at a gate: E4 and E5 are reported now as not yet established, and the report that establishes them, if it comes, needs no new charge.

6.8 The question Part II left to the example

Part II asked how much of the stale-suggestion rate is due to the record design, which the certified statistics remove, and how much to missed replies, which only better detection reduces [1, Section 9]. Running the two designs side by side answers it. Table 9 gives the counts for the gate window.

Every proposal of the event design was also a proposal of the window design. Of the window design's 1,839 stale proposals, 1,800 were made by it alone, each for an item whose closing note memory had seen and the window had forgotten; those are due to the record design, and

	Window design	Event design
Proposals logged	3,752	1,952
Stale proposals	1,839	39
also made by the other design	39	39
Keys on which the design erred, of 482	318	4

Table 9: Stale proposals and wrong decisions by design, days 28 to 55. Proposal counts are descriptions and cover every proposal of the window. The counts of keys are those of the lifetime receipts behind E3, which cover the keys, other than duplicates, whose daily decisions all fall in the window.

the certified statistics remove them. The other 39 were made by both designs and are due to the input events: 30 came from five keys whose replies the detector missed, and 9 from two duplicate keys of questions that had already been answered. Only better detection and better linking of paraphrases reduce those.

The lifetime receipts, which count keys, the unit at which E3 is stated, show the same division on a smaller set of keys. Of the 482 keys, the window design erred on 318 and the event design on 4, all four among the 318, and all four had lost their replies to the ordinary misses of version 1. The two parts of Table 9 cover different keys: the proposal counts include duplicate keys and keys created before the window, and the lifetime receipts leave both out.

7 The next release

After the gate the product side asks for two changes. The first is a snooze: on a proposal, "remind me in s days" keeps the engine from proposing the item for s days. The second extends the nag goal from two days to three. Both are memory decisions, and Part II's notes on Pattern 5 say what to do before such a release ships: check the classification of the goal change and check the store's statistics against the new goal set [1, Section 6.5]. The module `WMTMRelease` declares the new item machine and every statistic the release needs, and its requests answer both questions before any data from the release exist.

7.1 The item with a snooze

An item now has a fourth component, the days r of snooze left. Waiting d days lowers r to $\max(0, r - d)$. A snooze of s days sets r to $\max(r, s)$, so a second snooze can extend the first but never shorten it. The engine proposes an item when it is open, at most seven days old and not snoozed, $\neg c \wedge a \leq 7 \wedge r \leq 0$. The commitment record's query is unchanged, and the two nag goals read the age as before.

```
(DC.Module WMTMRelease
  (Profile (Dynamics Deterministic) (Values Crisp) (Carriers Symbolic))
  (Use RationalArith Families Statistics)
  (Import LiveOpenItems (Only Days joint-wait joint-reply joint-suggest joint-open))
  (Carrier Flag (Of Bool) (Enum False True))
  ;; an item: (age, closed, days since the closing reply, days of snooze left)
  (Carrier RawS (Of (Tuple Rational Bool Rational Rational))
    (Product Days Flag Days Days))
  (Map well-formed (From (Tuple Rational Bool Rational Rational)) (To Bool)
    (Cases ((Tuple $a $c $e $r) (Le $e $a))))
  (Carrier ItemS (Of (Tuple Rational Bool Rational Rational)) (Subset RawS well-formed))
  (Map wait (Parameters ($d Days)) (From ItemS) (To ItemS)
    (Cases ((Tuple $a $c $e $r) (Tuple (Add $a $d) $c (Add $e $d) (Max 0 (Sub $r $d))))))
```

```
(Map on-reply (From ItemS) (To ItemS)
  (Cases ((Tuple $a $c $e $r) (Tuple $a True (If $c $e 0) $r))))
;; "remind me in s days": the snooze ends s days from now, or later if it already did
(Map snooze (Parameters ($s Days)) (From ItemS) (To ItemS)
  (Cases ((Tuple $a $c $e $r) (Tuple $a $c $e (Max $r $s)))))
(Map suggest (From ItemS) (To Bool)
  (Cases ((Tuple $a $c $e $r) (And (Not $c) (And (Le $a 7) (Le $r 0))))))
(Map open (From ItemS) (To Bool) (Cases ((Tuple $a $c $e $r) (Not $c))))
(Map nag-two (From ItemS) (To Bool)
  (Cases ((Tuple $a $c $e $r) (And (Not $c) (Le $a 2)))))
(Map nag-three (From ItemS) (To Bool)
  (Cases ((Tuple $a $c $e $r) (And (Not $c) (Le $a 3)))))
(Machine ItemSnooze (States ItemS) (Observe suggest suggest) (Observe open open)
  (Action (wait $d) (wait $d) (For $d Days)) (Action reply on-reply)
  (Action (snooze $s) (snooze $s) (For $s Days)))
(Machine NagTwo (States ItemS) (Observe nag nag-two) (ActionsOf ItemSnooze))
(Machine NagThree (States ItemS) (Observe nag nag-three) (ActionsOf ItemSnooze))

;; the engine index of the log-only phase, carried over without a step for the snooze
(Map joint-carried (From ItemS) (To (Tuple Bool Bool Rational))
  (Cases ((Tuple $a $c $e $r)
    (Tuple $c (And (Not $c) (Lt 7 $a)) (If (And (Not $c) (Le $a 7)) $a 0)))))
(Statistic JointCarried (Of ItemSnooze) (Encode joint-carried)
  (Step (wait $d) (joint-wait $d) (For $d Days)) (Step reply joint-reply)
  (Read suggest joint-suggest) (Read open joint-open))
;; the same index as a representation of the new machine: what it can still answer now
(Representation CarriedIndex (Of ItemSnooze) (Encode joint-carried))
```

7.2 The deployed engine index is invalid input

`JointCarried` is the engine index of the deployed store, carried over to the new machine with its steps for waiting and replies and nothing for the snooze. The checker answers `InvalidInput` and names the missing step, before posing any obligation: a summary statistic must have an abstract update for every action family of its machine [1, Definition 3.1], and this one has none for the snooze. The status says that the declaration is not a statistic of the new machine. The request `CheckNow CarriedIndex`, on the same encoding declared as a representation of the new machine, shows that no step could be added to make it one. It is Refuted with the witness $x = (0, \text{false}, 0, 1)$ and $y = (0, \text{false}, 0, 0)$: two open items of age zero, the first snoozed for a day, which have the same code and different answers to the engine's query. A store that went on serving the engine from the deployed index after the release would therefore propose snoozed items, and the two requests establish this from the declarations alone, before a single snooze has been logged.

7.3 The record and the engine index after the release

```
;; the durable record: the age, whether the item is closed, and the snooze left
(Map rec (From ItemS) (To (Tuple Rational Bool Rational))
  (Cases ((Tuple $a $c $e $r) (Tuple $a $c $r))))
(Map rec-wait (Parameters ($d Days)) (From (Tuple Rational Bool Rational))
  (To (Tuple Rational Bool Rational))
  (Cases ((Tuple $a $c $r) (Tuple (Add $a $d) $c (Max 0 (Sub $r $d))))))
(Map rec-reply (From (Tuple Rational Bool Rational)) (To (Tuple Rational Bool Rational))
  (Cases ((Tuple $a $c $r) (Tuple $a True $r))))
(Map rec-snooze (Parameters ($s Days)) (From (Tuple Rational Bool Rational))
  (To (Tuple Rational Bool Rational))
  (Cases ((Tuple $a $c $r) (Tuple $a $c (Max $r $s)))))
(Map rec-suggest (From (Tuple Rational Bool Rational)) (To Bool)
```

```
  (Cases ((Tuple $a $c $r) (And (Not $c) (And (Le $a 7) (Le $r 0))))))
(Map rec-open (From (Tuple Rational Bool Rational)) (To Bool)
  (Cases ((Tuple $a $c $r) (Not $c))))
(Statistic RecordS (Of ItemSnooze) (Encode rec)
  (Step (wait $d) (rec-wait $d) (For $d Days)) (Step reply rec-reply)
  (Step (snooze $s) (rec-snooze $s) (For $s Days))
  (Read suggest rec-suggest) (Read open rec-open))

;; the coarsest engine index: closed; open but out of the engine's reach for good
;; (past seven days, or snoozed past the seventh day); or live, with age and snooze
(Map joint (From ItemS) (To (Tuple Bool Bool Rational Rational))
  (Cases ((Tuple $a $c $e $r)
    (Tuple $c (And (Not $c) (Lt 7 (Add $a $r)))
           (If (And (Not $c) (Le (Add $a $r) 7)) $a 0)
           (If (And (Not $c) (Le (Add $a $r) 7)) $r 0)))))
(Map joint-s-wait (Parameters ($d Days)) (From (Tuple Bool Bool Rational Rational))
  (To (Tuple Bool Bool Rational Rational))
  (Cases ((Tuple $c $t $v $u)
    (Tuple $c (And (Not $c) (Or $t (Lt 7 (Add $v $d))))
           (If (And (Not $c) (Not (Or $t (Lt 7 (Add $v $d))))) (Add $v $d) 0)
           (If (And (Not $c) (Not (Or $t (Lt 7 (Add $v $d)))))
               (Max 0 (Sub $u $d)) 0)))))
(Map joint-s-reply (From (Tuple Bool Bool Rational Rational))
  (To (Tuple Bool Bool Rational Rational))
  (Cases ((Tuple $c $t $v $u) (Tuple True False 0 0))))
(Map joint-s-snooze (Parameters ($s Days)) (From (Tuple Bool Bool Rational Rational))
  (To (Tuple Bool Bool Rational Rational))
  (Cases ((Tuple $c $t $v $u)
    (Tuple $c (And (Not $c) (Or $t (Lt 7 (Add $v (Max $u $s)))))
           (If (And (Not $c) (Not (Or $t (Lt 7 (Add $v (Max $u $s)))))) $v 0)
           (If (And (Not $c) (Not (Or $t (Lt 7 (Add $v (Max $u $s))))))
               (Max $u $s) 0)))))
(Map joint-s-suggest (From (Tuple Bool Bool Rational Rational)) (To Bool)
  (Cases ((Tuple $c $t $v $u) (And (Not $c) (And (Not $t) (Le $u 0))))))
(Map joint-s-open (From (Tuple Bool Bool Rational Rational)) (To Bool)
  (Cases ((Tuple $c $t $v $u) (Not $c))))
(Statistic JointS (Of ItemSnooze) (Encode joint)
  (Step (wait $d) (joint-s-wait $d) (For $d Days)) (Step reply joint-s-reply)
  (Step (snooze $s) (joint-s-snooze $s) (For $s Days))
  (Read suggest joint-s-suggest) (Read open joint-s-open))
```

`RecordS` is the durable record of the release: the age, the closed flag and the snooze left. It is Conditional on cvc5 with eight obligations: closure and update for waiting, replies and snoozes, and reads for both queries. In the stored records this means one more timestamp per key beside the creation time and the time from which the item is closed. Since a snooze sets the snooze left to the larger of the old and the new, the timestamp to keep is the latest end of any snooze so far.

`JointS` is the engine index of the release. It keeps three kinds of value: closed; open but out of the engine's reach for good; and live, with its age and its snooze. An open item is out of reach when $a + r > 7$, because by the time its snooze ends it will be past seven days. The statistic is Conditional on cvc5 with eight obligations, and it is the coarsest for the engine and the commitment record together.

The oracle checks that minimality in Z3 in the way Part II checks its own statistics [1, Section 3.3]: for any two different codes it gives a context, a sequence of actions followed by a query, that separates them. Two live codes with different snoozes are separated by waiting out the shorter snooze, after which only one of the two is proposed. Two live codes with the same snooze and different ages are separated by waiting until the younger is seven days old. A live code and the out-of-reach class are separated by waiting out the live item's snooze, and

the out-of-reach class stays out of reach under every wait and every snooze. The commitment query separates closed items from open ones. By Part II's Proposition 3.4, a statistic whose different values are all separated by some context and query is the coarsest.

In plain terms. The engine index of the release keeps the age and the snooze of an open item for as long as the item can still come back to the engine, and nothing more once it cannot. Any two items that it stores differently are, at some later moment, treated differently by the engine or by the commitment record, so no smaller index serves both.

The out-of-reach class merges open items that differ in age and in snooze, and its members include items that a person snoozed in good faith. An item five days old that is snoozed for three days leaves the engine for good, while the commitment record still holds it open. The randomized test of the oracle met such states in its runs. Whether that is what "remind me in three days" should mean is a product question, and the declarations raise it before the release ships. The release can accept it, cap a snooze at $7 - a$ days, or extend the engine's query to snoozed items past seven days, which would be a goal change to classify as in Part II's Pattern 5.

7.4 The three-day nag

```
;; the nag index: saturation at two days; the snooze does not change it
(Map sat-two (From ItemS) (To (Tuple Bool Bool Rational))
  (Cases ((Tuple $a $c $e $r)
    (Tuple (Not $c) (And (Not $c) (Lt 2 $a)) (If (And (Not $c) (Le $a 2)) $a 0)))))
(Map sat-two-wait (Parameters ($d Days)) (From (Tuple Bool Bool Rational))
  (To (Tuple Bool Bool Rational))
  (Cases ((Tuple $o $t $v)
    (Tuple $o (And $o (Or $t (Lt 2 (Add $v $d))))
           (If (And $o (Not (Or $t (Lt 2 (Add $v $d))))) (Add $v $d) 0)))))
(Map sat-close (From (Tuple Bool Bool Rational)) (To (Tuple Bool Bool Rational))
  (Cases ((Tuple $o $t $v) (Tuple False False 0))))
(Map sat-keep (Parameters ($s Days)) (From (Tuple Bool Bool Rational))
  (To (Tuple Bool Bool Rational)) (Cases ((Tuple $o $t $v) (Tuple $o $t $v))))
(Map sat-nag (From (Tuple Bool Bool Rational)) (To Bool)
  (Cases ((Tuple $o $t $v) (And $o (Not $t)))))
(Statistic NagTwoIndex (Of NagTwo) (Encode sat-two)
  (Step (wait $d) (sat-two-wait $d) (For $d Days)) (Step reply sat-close)
  (Step (snooze $s) (sat-keep $s) (For $s Days)) (Read nag sat-nag))
;; the planned goal reads three days; the two-day index stores ages past two days alike
(Representation TwoForThree (Of NagThree) (Encode sat-two))
(Map sat-three (From ItemS) (To (Tuple Bool Bool Rational))
  (Cases ((Tuple $a $c $e $r)
    (Tuple (Not $c) (And (Not $c) (Lt 3 $a)) (If (And (Not $c) (Le $a 3)) $a 0)))))
(Map sat-three-wait (Parameters ($d Days)) (From (Tuple Bool Bool Rational))
  (To (Tuple Bool Bool Rational))
  (Cases ((Tuple $o $t $v)
    (Tuple $o (And $o (Or $t (Lt 3 (Add $v $d))))
           (If (And $o (Not (Or $t (Lt 3 (Add $v $d))))) (Add $v $d) 0)))))
(Statistic NagThreeIndex (Of NagThree) (Encode sat-three)
  (Step (wait $d) (sat-three-wait $d) (For $d Days)) (Step reply sat-close)
  (Step (snooze $s) (sat-keep $s) (For $s Days)) (Read nag sat-nag))
;; rebuilding the new index from the durable record: the record is a statistic of
;; the machine that observes the new index, and its read map is the rebuild
(Machine NewIndexView (States ItemS) (Observe index sat-three) (ActionsOf ItemSnooze))
(Map rebuild-three (From (Tuple Rational Bool Rational)) (To (Tuple Bool Bool Rational))
  (Cases ((Tuple $a $c $r)
    (Tuple (Not $c) (And (Not $c) (Lt 3 $a)) (If (And (Not $c) (Le $a 3)) $a 0)))))
(Statistic RebuildFromRecords (Of NewIndexView) (Encode rec)
  (Step (wait $d) (rec-wait $d) (For $d Days)) (Step reply rec-reply)
  (Step (snooze $s) (rec-snooze $s) (For $s Days)) (Read index rebuild-three))
```

```
;; the engine index is rebuilt from the durable record in the same way
(Machine EngineIndexView (States ItemS) (Observe index joint) (ActionsOf ItemSnooze))
(Map rebuild-joint (From (Tuple Rational Bool Rational))
  (To (Tuple Bool Bool Rational Rational))
  (Cases ((Tuple $a $c $r)
    (Tuple $c (And (Not $c) (Lt 7 (Add $a $r)))
           (If (And (Not $c) (Le (Add $a $r) 7)) $a 0)
           (If (And (Not $c) (Le (Add $a $r) 7)) $r 0)))))
(Statistic RebuildEngineIndex (Of EngineIndexView) (Encode rec)
  (Step (wait $d) (rec-wait $d) (For $d Days)) (Step reply rec-reply)
  (Step (snooze $s) (rec-snooze $s) (For $s Days)) (Read index rebuild-joint))

;; the store after the release
(Carrier Person (Of Symbol) (All))
(Carrier Topic  (Of Symbol) (All))
(Carrier ItemKey (Of (Tuple Symbol Symbol)) (Product Person Topic))
(Store ItemsS (Keys ItemKey) (Of ItemSnooze) (Global wait))
(Keyed RecordsByKey (Of ItemsS) (Item RecordS))
(Keyed EngineIndexS (Of ItemsS) (Item JointS))
(Keyed NagIndexS (Of ItemsS) (Item NagThreeIndex) (View NagThree)))
```

The two-day index of the deployed store is still a statistic of the two-day goal on the new item: `NagTwoIndex` is Conditional on cvc5, and its snooze step leaves the index unchanged, since the nag goal does not read the snooze. As a representation of the three-day goal it is Refuted, with the witness of Part II's Pattern 5: two open items of ages 5/2 and 7/2, both past two days and therefore stored alike, of which only the first is within three days. The saturation at three days, `NagThreeIndex`, is Conditional on cvc5 as well. The classification of the goal change itself needs no new request: Part II found that extending the nag goal from two days to three changes its content, since at a given moment neither goal's answers determine the other's [1, Section 6.5]. Across moments the two are related: the two-day goal's answer about an item now is the three-day goal's answer about it a day later, if nothing happens to the item in between.

7.5 Rebuilding the indexes from the records

The release has to replace the two-day index by the three-day index for every key that exists when it ships. At that moment, items between two and three days old have lost their ages in the two-day index, which is exactly what the three-day goal reads. The durable record kept them, and the module states the rebuild as a summary statistic. The machine `NewIndexView` has the actions of the item and one query, the three-day index itself. The record is a statistic of that machine whose read map, `rebuild-three`, is the rebuild: identity (R) says that rebuilding the record of an item gives the item's index, and (C) and (U) say that the record keeps pace with every action. `RebuildFromRecords` is Conditional on cvc5 with seven obligations.

Part II's Proposition 3.2 then applies twice. Applied to `RebuildFromRecords`, it gives that after every history the rebuild of the stored record equals the three-day index of the item's state. Applied to `NagThreeIndex`, whose update identity holds for every action, the same argument gives that the index maintained step by step from the item's creation is also the three-day index of the item's state. The rebuilt index is therefore the index the key would have if the three-day index had been maintained from the start. `RebuildEngineIndex` states and checks the same for the engine index of the release, with `JointS` in the place of `NagThreeIndex`. Only indexes that are functions of the stored age, closed flag and snooze left can be rebuilt this way.

In plain terms. Rebuilding the three-day index from the stored timestamps gives every key exactly the index it would have had if the three-day index had been kept from the start,

whatever happened to the key before the release. The solver checks this once, for all items and all histories.

Two differential tests in the oracle run these maps on data. At the end of day 55, for each of the 1,362 keys of the deployment, the three-day index and the engine index rebuilt from the key's record equal the indexes obtained by stepping the key's whole history through the release's maps. In 400 random runs of twelve steps each, 4,800 steps of waits, replies and snoozes, the record and the engine index decide as a direct implementation that keeps absolute times: the creation time, the clock, whether a reply has come and when the snooze ends. Their codes equal the encodings of the replayed items. Table 10 lists the verdicts of the release's requests.

Request	Status	Obligations or witness
`CheckStatistic JointCarried`	InvalidInput	missing: the step for `snooze`
`CheckNow CarriedIndex`	Refuted	$(0, \text{false}, 0, 1)$ and $(0, \text{false}, 0, 0)$: the same code, and only the second is proposed
`CheckStatistic RecordS`	Conditional	eight
`CheckStatistic JointS`	Conditional	eight; coarsest for the engine and the commitment record
`CheckStatistic NagTwoIndex`	Conditional	seven; the snooze leaves it unchanged
`CheckNow TwoForThree`	Refuted	open items of ages 5/2 and 7/2
`CheckStatistic NagThreeIndex`	Conditional	seven
`CheckStatistic RebuildFromRecords`	Conditional	seven; the three-day index from the record
`CheckStatistic RebuildEngineIndex`	Conditional	seven; the engine index from the record
`CheckStatistic EngineIndexS`	Conditional	by lifting `JointS`
`CheckStatistic NagIndexS`	Conditional	by lifting `NagThreeIndex` to the view `NagThree`

Table 10: The release, checked before it ships. Conditional: cvc5 found every obligation unsatisfiable and Z3 agrees; no proof is checked.

```
!(dc.run WMTMRelease (CheckStatistic JointCarried (Method Symbolic)))
!(dc.run WMTMRelease (CheckNow CarriedIndex))
!(dc.run WMTMRelease (CheckStatistic RecordS (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic JointS (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic NagTwoIndex (Method Symbolic)))
!(dc.run WMTMRelease (CheckNow TwoForThree))
!(dc.run WMTMRelease (CheckStatistic NagThreeIndex (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic RebuildFromRecords (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic RebuildEngineIndex (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic EngineIndexS (Method Lift)))
!(dc.run WMTMRelease (CheckStatistic NagIndexS (Method Lift)))
```

7.6 What the release brings to the gate

The release changes the engine's query, so it is a new candidate at the gate, and the certificates of Table 10 are its static evidence. The three claims of the gate's absolute budget carry over to it without new data, each by an argument of its own. The new query asks for everything the old one asked and one more condition, so on the same journal the release can only remove proposals: every reply receipt that is stale under the release was stale before, and since the upper end of E1's interval rises with the count of stale receipts, it remains an upper bound for the release's rate under the same assumptions. E6 concerns the records and does not depend on the query. E2 is about keys at their first proposals, and these do not change. A snooze is given only on a proposal, so until a key's first proposal its snooze left is zero and the two

queries agree: every key is first proposed at the same cycle as before, and `KeysGate` has the same rows and labels under the release. The oracle checks this on the frozen journal. It steps every key through the deployed engine index and through the release's, giving a snooze of one to four days on a random 30 percent of the release's proposals, 900 snoozes in all. All 1,362 keys are first proposed at the same cycle as in the deployment, while the snoozes remove 411 of the 1,952 proposals of the gate window.

The argument for E2 rests on a detail of the product design. A release that let a person snooze an item before the engine proposes it, from the item's page say, could keep a key from being proposed at all, and the keys the engine proposes would then be a selection that depends on the policy. In the extreme case, a policy that withheld every key that is not a duplicate would propose only the five duplicate keys of the gate window: a subset of the log-only proposals, with a share of duplicates of five in five instead of five in 453. A rate bounded on one policy's selection of keys says nothing about another's, so such a release would need new key receipts, or a declared argument, before E2 could be claimed for it.

What is new in this release is a way to fail that the declarations exclude and an implementation might not: proposing an item before its snooze ends. The replay of Section 3.5, run on the release's log with the release's maps, is the check for that, as it was for the first promotion.

8 What the example teaches implementers

8.1 Declarations

The declarations of the store fill three modules in a file of 357 lines, with 33 requests. Four of the requests rerun checks that Part II made on the declarations the example imports from it, two lift Part II's statistics to the new store, and the other 27 are new. The declarations did not change during the incident, and their certificates were as valid on day 23 as on day 0, because the regression was in the input events and every statistic is sufficient relative to the events it is given. The checking happened at two moments, when the store was declared and when the release was planned, and the whole oracle, with every check of this text, runs in about twelve seconds on one core. An implementer should expect this distribution of effort: the declarations are short and stable, and most of the work goes into the receipts.

8.2 Receipts

Receipts should be designed before they are collected. For each claim, choose the unit whose average is what the claim is about, write the receipt at a moment fixed in advance, and give it labels that the unit's own events decide. A one-sided claim then holds for the average of the units' conditional rates, with no assumption of independence or of a common rate; reading the same interval as an estimate of one rate needs both assumptions in addition. For the goal engine that meant replies, keys, key lifetimes, first closings and audited items, and it meant keeping the per-proposal labels only as descriptions. Release the receipts of a cycle together, in an order that does not depend on their labels. Give every stream its population, selection rule and policy, since an estimate made under one policy carries over to another only with an argument; Section 7.6 carries the gate's claims over to the release, each with its own argument, and shows how a snooze given before the first proposal would break the argument for E2. For each source of labels, write down what it cannot see: the review of proposals misses every false closing, since a falsely closed item is never proposed, the lifetime labels miss an error that both designs make at once, and the random audit sees memory at one age only and accumulates slowly. Part I's receipt formats [2, Appendix C.4] should therefore gain reply, key, lifetime, closure and audit receipts next to the suggestion receipt.

8.3 Monitors

A monitor on a live system should be a changepoint monitor. A process started at the first observation tests whether a stream has been consistent with the accepted rate all along, and after weeks of health it reacts late to a change, as Section 5.3 showed. The Shiryaev–Roberts form gives up the bound on the probability of any false alarm and offers instead a bound on the mean time to one and on its probability within any fixed number of receipts. That is acceptable when an alarm leads to a confirmation on data the alarm did not use, as the audit of the replies received after the alarm did here. The response to an alarm, the confirmation rule and the action it licenses, should be fixed before the monitor runs, and the monitor restarted once the repair is in place.

8.4 Gates

Each Estimated claim at a gate rests on the receipts it counts and on hypotheses about where their labels come from. The bundle should record those hypotheses with the claims and name the independent check of each; here the hypothesis was that the review finds the errors it labels, and the random audit was its check. The decision to promote before the check is decisive is a judgment the bundle should record, together with the conditions under which it is reversed. State the absolute error budget beside every comparative claim, since a comparison with a poor design bounds no absolute harm. The sizes a claim needs should be planned before the gate window opens, for a stated probability of success rather than at the plug-in size. Here they showed that a four-week window with a one-third audit cannot establish a target of one in twenty for the audit's rates. Because the intervals are confidence sequences, the audit's claims can be carried from one gate to the next on the level already spent.

8.5 Releases and durable records

Every release that changes the store's actions or its consumers should rerun `CheckStatistic` on the release's machine before it ships. An `InvalidInput` for a missing step says that an index must be extended or replaced, and a `CheckNow` witness on its encoding says when it must be replaced. Here the deployed engine index had to be replaced, and the two-day nag index needed only a step for the snooze, one that leaves it unchanged. The coarsest replacement can surface product decisions, such as the snoozed items that leave the engine for good. The durable records should keep the event statistic and what the release's actions add to it, here the latest end of any snooze, because only indexes that are functions of the record can be rebuilt from it. Stating the rebuild as a statistic of the machine that observes the index turns "the index can be rebuilt from the records" into a claim that a solver decides.

9 Limits

The parameters of Table 1 are plausible choices and not measurements of WMTM. The simulated review and audit make no mistakes, and the input errors are independent by construction. In a live deployment reviewers overlook things, and misses can be correlated, by topic or by person. Correlated misses leave the gate's one-sided claims valid, since these hold for the average of the receipts' conditional rates, but they make the two-sided intervals too narrow when these are read as estimates of one rate, and they make the planning sizes, which assume one rate, a rough guide.

The gate's four established claims rest on the review. A missed reply that the reviewer also overlooks produces no receipt, so the reply streams undercount misses at the review's own miss

rate, and the same holds for duplicates and false closings that the review does not recognize. The audit bounds that effect only as fast as it accumulates, and Section 6.4 records the decision to promote before the audit is decisive.

The changepoint monitor's guarantee bounds the mean time to a false alarm and the probability of one over a fixed number of receipts, not over an unbounded future. It assumes the conditional rate bound of this example's Proposition 5.1 for the receipts in the order in which the monitor reads them; the receipts of a cycle are released in the order of the replies' times, which does not depend on their labels.

The planning sizes of Section 6.5 use the simulation's rates. A live team does not know the true rates; it plans with pessimistic ones, and the times become longer.

The parser, the reply detector and the topic labeler form the abstraction function, which is code under test and which the conformance replay does not test; the language model is outside the store altogether. Their behavior enters only through receipts, as in Parts I and II.

The monitors average over three declared alternatives. A continuous mixture over all rates above the accepted one, which Part II computed in Python, cannot yet be declared [1, Section 10].

The symbolic verdicts are Conditional on cvc5. Z3 agrees with each of them on encodings written by hand, which is evidence and not a proof, and no proof is checked; Part II discusses which of these verdicts could be backed by checked proofs today [1, Section 7.2]. The engine's queries accept results of this kind, and a consumer with the frame's custody would not.

10 Conclusion

The store at the center of this example is small: a few timestamps per key and the indexes computed from them. A solver established once, for every state the store can reach, that the store answers what its consumers ask, and established it again for the release before it shipped. The parser, the detector and the reviewer, which no solver reaches, were measured from receipts at units chosen so that each claim bounds an average that keeps its meaning once the engine is live. A changepoint monitor watched those receipts, and an audit of replies that the alarm had not used confirmed its alarm before anyone acted on it. Three of the five points at which the example needed more than Part II's protocol concern the receipts, the monitor and the gate; the other two concern releases, which the protocol did not cover. The example also answered the question Part II left open: in the gate window the record design accounts for 1,800 of the window design's 1,839 stale proposals, and the input events for the other 39.

I find it encouraging that the declarations needed no change once they were written. The incident left every certificate valid, and the release added certificates without touching the old ones. The distinction calculus asks of every memory which distinctions it must keep and for whom; for this store that question took three short modules and a few seconds of solver time to answer, and most of the effort went into the receipts. The next step is to run the same requests on streams frozen from the live system, where the review makes mistakes and the parameters are no longer chosen.

References

[1] *Omega Memory and Goals in MeTTaDinC, Part II: Live Memory through Summary Statistics.* Companion to the MeTTaDinC specification draft, 4 November 2026.

[2] *Omega Memory and Goals in MeTTaDinC: A Practical Implementation Guide.* Companion to the MeTTaDinC specification draft, 4 November 2026.

[3] *MeTTaDinC: An Executable Language for Observer-Relative Abstraction, Evidence, and Composition.* Specification draft, 4 November 2026.

[4] *MeTTaDinC in Practice: Four Worked Examples.* Companion to the MeTTaDinC specification draft, 4 November 2026.

[5] A. Suarez. *WMTM: Current State and Plan.* Report, 21 October 2026.

[6] B. Goertzel. *GOLEM-Iter: Goal-Governed and Regeneratively Anchored Self-Reprogramming for OmegaClaw Agents.* Draft v0.4, August 2026.

[7] E. S. Page. Continuous inspection schemes. *Biometrika* 41(1–2):100–115, 1954.

[8] A. N. Shiryaev. On optimum methods in quickest detection problems. *Theory of Probability and Its Applications* 8(1):22–46, 1963.

[9] S. W. Roberts. A comparison of some control chart procedures. *Technometrics* 8:411–430, 1966.

[10] G. Lorden. Procedures for reacting to a change in distribution. *Annals of Mathematical Statistics* 42(6):1897–1908, 1971.

[11] M. Pollak. Optimal detection of a change in distribution. *Annals of Statistics* 13(1):206–227, 1985.

[12] J. Shin, A. Ramdas and A. Rinaldo. E-detectors: a nonparametric framework for sequential change detection. *New England Journal of Statistics in Data Science* 2(2):229–260, 2024.

[13] J. Ville. *Étude critique de la notion de collectif.* Gauthier-Villars, 1939.

[14] A. Ramdas, P. Grünwald, V. Vovk and G. Shafer. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science* 38(4):576–601, 2023.

[15] H. Robbins. Statistical methods related to the law of the iterated logarithm. *Annals of Mathematical Statistics* 41(5):1397–1409, 1970.

[16] S. R. Howard, A. Ramdas, J. McAuliffe and J. Sekhon. Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics* 49(2):1055–1080, 2021.

[17] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12(2):153–157, 1947.

[18] L. de Moura and N. Bjørner. Z3: an efficient SMT solver. In *TACAS 2008*, LNCS 4963, 337–340.

[19] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli and Y. Zohar. cvc5: a versatile and industrial-strength SMT solver. In *TACAS 2022*, LNCS 13243, 415–442.

A The module file

The file `wmtm_example.metta` follows in full, exactly as it is shipped in the bundle. It holds three modules; the excerpts printed in the text are the regions between the lines `;; @excerpt` and `;; @end`. Comments marked `PROPOSED` flag forms that need a registry patch (Appendix B). The modules `WMTMStore` and `WMTMRelease` import from Part II's module `LiveOpenItems`, which is shipped as `live_memory.metta`; that file in turn imports from Part I's `memory_goals.metta`.

```
;; WMTM's open-item store: the modules of the worked example of Part II of the guide,
;; 4 November 2026.  The deployment is simulated: eight weeks of the goal engine's
;; log-only phase, generated by models/wx_sim.py with the seed 20261104.  Forms marked
;; PROPOSED need registry patches: those of Part II (ls_registry_patch.json, among them
;; the View field of Keyed and the Population, Selection and Policy fields of Stream) and
;; of its predecessors, and the Scheme and MeanTimeToFalseAlarm arguments of Detect
;; (wx_registry_patch.json).  The lines ";; @excerpt <name>" and ";; @end" delimit the
;; excerpts printed in the guide.

;; ============================================================ 1. the store as deployed
;; @excerpt wmtm-store
(DC.Module WMTMStore
  (Profile (Dynamics Deterministic) (Values Crisp) (Carriers Symbolic))
  (Use RationalArith Families Statistics)              ; PROPOSED (Part II)
  ;; the item machine and its statistics, unchanged from Part II
  (Import LiveOpenItems
    (Only Days Item ItemTime EngineTime EventStat JointStat WindowView))

  ;; the nag goal: follow up on an open item while it is at most two days old
  (Map nag (From Item) (To Bool) (Cases ((Tuple $a $c $e) (And (Not $c) (Le $a 2)))))
  (Machine NagTime (States Item) (Observe nag nag) (ActionsOf ItemTime))
  ;; its index: open, past two days, and the age while it is at most two days
  (Map sat-two (From Item) (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $a $c $e)
      (Tuple (Not $c) (And (Not $c) (Lt 2 $a)) (If (And (Not $c) (Le $a 2)) $a 0)))))
  (Map sat-two-wait (Parameters ($d Days)) (From (Tuple Bool Bool Rational))
    (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $o $t $v)
      (Tuple $o (And $o (Or $t (Lt 2 (Add $v $d))))
             (If (And $o (Not (Or $t (Lt 2 (Add $v $d))))) (Add $v $d) 0)))))
  (Map sat-two-close (From (Tuple Bool Bool Rational)) (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $o $t $v) (Tuple False False 0))))
  (Map sat-two-nag (From (Tuple Bool Bool Rational)) (To Bool)
    (Cases ((Tuple $o $t $v) (And $o (Not $t)))))
  (Statistic NagIndex (Of NagTime) (Encode sat-two)
    (Step (wait $d) (sat-two-wait $d) (For $d Days)) (Step reply sat-two-close)
    (Read nag sat-two-nag))

  ;; the store, keyed by the engine's deduplication key: the person and the topic label
  (Carrier Person (Of Symbol) (All))
  (Carrier Topic  (Of Symbol) (All))
  (Carrier ItemKey (Of (Tuple Symbol Symbol)) (Product Person Topic))
  (Store Items (Keys ItemKey) (Of ItemTime) (Global wait))      ; PROPOSED (Part II)
  (Keyed Records (Of Items) (Item EventStat))                  ; durable
  (Keyed EngineIndex (Of Items) (Item JointStat))              ; rebuilt from Records
  (Keyed NagIndexByKey (Of Items) (Item NagIndex)              ; rebuilt from Records
    (View NagTime)))                                           ; PROPOSED View (Part II)
;; @end

;; @excerpt wmtm-store-requests
!(dc.run WMTMStore (CheckStatistic EventStat (Method Symbolic)))
!(dc.run WMTMStore (CheckStatistic JointStat (Method Symbolic)))
!(dc.run WMTMStore (CheckStatistic NagIndex (Method Symbolic)))
!(dc.run WMTMStore (CheckNow WindowView))
!(dc.run WMTMStore (CheckUpdates WindowView))
!(dc.run WMTMStore (CheckStatistic Records (Method Lift)))
!(dc.run WMTMStore (CheckStatistic EngineIndex (Method Lift)))
!(dc.run WMTMStore (CheckStatistic NagIndexByKey (Method Lift)))
;; @end

;; ====================================== 2. the release: snooze and a three-day nag
;; @excerpt wmtm-release
```

```
(DC.Module WMTMRelease
  (Profile (Dynamics Deterministic) (Values Crisp) (Carriers Symbolic))
  (Use RationalArith Families Statistics)
  (Import LiveOpenItems (Only Days joint-wait joint-reply joint-suggest joint-open))
  (Carrier Flag (Of Bool) (Enum False True))
  ;; an item: (age, closed, days since the closing reply, days of snooze left)
  (Carrier RawS (Of (Tuple Rational Bool Rational Rational))
    (Product Days Flag Days Days))
  (Map well-formed (From (Tuple Rational Bool Rational Rational)) (To Bool)
    (Cases ((Tuple $a $c $e $r) (Le $e $a))))
  (Carrier ItemS (Of (Tuple Rational Bool Rational Rational)) (Subset RawS well-formed))
  (Map wait (Parameters ($d Days)) (From ItemS) (To ItemS)
    (Cases ((Tuple $a $c $e $r) (Tuple (Add $a $d) $c (Add $e $d) (Max 0 (Sub $r $d))))))
  (Map on-reply (From ItemS) (To ItemS)
    (Cases ((Tuple $a $c $e $r) (Tuple $a True (If $c $e 0) $r))))
  ;; "remind me in s days": the snooze ends s days from now, or later if it already did
  (Map snooze (Parameters ($s Days)) (From ItemS) (To ItemS)
    (Cases ((Tuple $a $c $e $r) (Tuple $a $c $e (Max $r $s)))))
  (Map suggest (From ItemS) (To Bool)
    (Cases ((Tuple $a $c $e $r) (And (Not $c) (And (Le $a 7) (Le $r 0))))))
  (Map open (From ItemS) (To Bool) (Cases ((Tuple $a $c $e $r) (Not $c))))
  (Map nag-two (From ItemS) (To Bool)
    (Cases ((Tuple $a $c $e $r) (And (Not $c) (Le $a 2)))))
  (Map nag-three (From ItemS) (To Bool)
    (Cases ((Tuple $a $c $e $r) (And (Not $c) (Le $a 3)))))
  (Machine ItemSnooze (States ItemS) (Observe suggest suggest) (Observe open open)
    (Action (wait $d) (wait $d) (For $d Days)) (Action reply on-reply)
    (Action (snooze $s) (snooze $s) (For $s Days)))
  (Machine NagTwo (States ItemS) (Observe nag nag-two) (ActionsOf ItemSnooze))
  (Machine NagThree (States ItemS) (Observe nag nag-three) (ActionsOf ItemSnooze))

  ;; the engine index of the log-only phase, carried over without a step for the snooze
  (Map joint-carried (From ItemS) (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $a $c $e $r)
      (Tuple $c (And (Not $c) (Lt 7 $a)) (If (And (Not $c) (Le $a 7)) $a 0)))))
  (Statistic JointCarried (Of ItemSnooze) (Encode joint-carried)
    (Step (wait $d) (joint-wait $d) (For $d Days)) (Step reply joint-reply)
    (Read suggest joint-suggest) (Read open joint-open))
  ;; the same index as a representation of the new machine: what it can still answer now
  (Representation CarriedIndex (Of ItemSnooze) (Encode joint-carried))
;; @end

;; @excerpt wmtm-release-record
  ;; the durable record: the age, whether the item is closed, and the snooze left
  (Map rec (From ItemS) (To (Tuple Rational Bool Rational))
    (Cases ((Tuple $a $c $e $r) (Tuple $a $c $r))))
  (Map rec-wait (Parameters ($d Days)) (From (Tuple Rational Bool Rational))
    (To (Tuple Rational Bool Rational))
    (Cases ((Tuple $a $c $r) (Tuple (Add $a $d) $c (Max 0 (Sub $r $d))))))
  (Map rec-reply (From (Tuple Rational Bool Rational)) (To (Tuple Rational Bool Rational))
    (Cases ((Tuple $a $c $r) (Tuple $a True $r))))
  (Map rec-snooze (Parameters ($s Days)) (From (Tuple Rational Bool Rational))
    (To (Tuple Rational Bool Rational))
    (Cases ((Tuple $a $c $r) (Tuple $a $c (Max $r $s)))))
  (Map rec-suggest (From (Tuple Rational Bool Rational)) (To Bool)
    (Cases ((Tuple $a $c $r) (And (Not $c) (And (Le $a 7) (Le $r 0))))))
  (Map rec-open (From (Tuple Rational Bool Rational)) (To Bool)
    (Cases ((Tuple $a $c $r) (Not $c))))
  (Statistic RecordS (Of ItemSnooze) (Encode rec)
    (Step (wait $d) (rec-wait $d) (For $d Days)) (Step reply rec-reply)
    (Step (snooze $s) (rec-snooze $s) (For $s Days))
    (Read suggest rec-suggest) (Read open rec-open))

  ;; the coarsest engine index: closed; open but out of the engine's reach for good
  ;; (past seven days, or snoozed past the seventh day); or live, with age and snooze
  (Map joint (From ItemS) (To (Tuple Bool Bool Rational Rational))
    (Cases ((Tuple $a $c $e $r)
      (Tuple $c (And (Not $c) (Lt 7 (Add $a $r)))
             (If (And (Not $c) (Le (Add $a $r) 7)) $a 0)
             (If (And (Not $c) (Le (Add $a $r) 7)) $r 0)))))
  (Map joint-s-wait (Parameters ($d Days)) (From (Tuple Bool Bool Rational Rational))
    (To (Tuple Bool Bool Rational Rational))
    (Cases ((Tuple $c $t $v $u)
```

```
      (Tuple $c (And (Not $c) (Or $t (Lt 7 (Add $v $d))))
             (If (And (Not $c) (Not (Or $t (Lt 7 (Add $v $d))))) (Add $v $d) 0)
             (If (And (Not $c) (Not (Or $t (Lt 7 (Add $v $d)))))
                 (Max 0 (Sub $u $d)) 0)))))
  (Map joint-s-reply (From (Tuple Bool Bool Rational Rational))
    (To (Tuple Bool Bool Rational Rational))
    (Cases ((Tuple $c $t $v $u) (Tuple True False 0 0))))
  (Map joint-s-snooze (Parameters ($s Days)) (From (Tuple Bool Bool Rational Rational))
    (To (Tuple Bool Bool Rational Rational))
    (Cases ((Tuple $c $t $v $u)
      (Tuple $c (And (Not $c) (Or $t (Lt 7 (Add $v (Max $u $s)))))
             (If (And (Not $c) (Not (Or $t (Lt 7 (Add $v (Max $u $s)))))) $v 0)
             (If (And (Not $c) (Not (Or $t (Lt 7 (Add $v (Max $u $s))))))
                 (Max $u $s) 0)))))
  (Map joint-s-suggest (From (Tuple Bool Bool Rational Rational)) (To Bool)
    (Cases ((Tuple $c $t $v $u) (And (Not $c) (And (Not $t) (Le $u 0))))))
  (Map joint-s-open (From (Tuple Bool Bool Rational Rational)) (To Bool)
    (Cases ((Tuple $c $t $v $u) (Not $c))))
  (Statistic JointS (Of ItemSnooze) (Encode joint)
    (Step (wait $d) (joint-s-wait $d) (For $d Days)) (Step reply joint-s-reply)
    (Step (snooze $s) (joint-s-snooze $s) (For $s Days))
    (Read suggest joint-s-suggest) (Read open joint-s-open))
;; @end

;; @excerpt wmtm-release-nag
  ;; the nag index: saturation at two days; the snooze does not change it
  (Map sat-two (From ItemS) (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $a $c $e $r)
      (Tuple (Not $c) (And (Not $c) (Lt 2 $a)) (If (And (Not $c) (Le $a 2)) $a 0)))))
  (Map sat-two-wait (Parameters ($d Days)) (From (Tuple Bool Bool Rational))
    (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $o $t $v)
      (Tuple $o (And $o (Or $t (Lt 2 (Add $v $d))))
             (If (And $o (Not (Or $t (Lt 2 (Add $v $d))))) (Add $v $d) 0)))))
  (Map sat-close (From (Tuple Bool Bool Rational)) (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $o $t $v) (Tuple False False 0))))
  (Map sat-keep (Parameters ($s Days)) (From (Tuple Bool Bool Rational))
    (To (Tuple Bool Bool Rational)) (Cases ((Tuple $o $t $v) (Tuple $o $t $v))))
  (Map sat-nag (From (Tuple Bool Bool Rational)) (To Bool)
    (Cases ((Tuple $o $t $v) (And $o (Not $t)))))
  (Statistic NagTwoIndex (Of NagTwo) (Encode sat-two)
    (Step (wait $d) (sat-two-wait $d) (For $d Days)) (Step reply sat-close)
    (Step (snooze $s) (sat-keep $s) (For $s Days)) (Read nag sat-nag))
  ;; the planned goal reads three days; the two-day index stores ages past two days alike
  (Representation TwoForThree (Of NagThree) (Encode sat-two))
  (Map sat-three (From ItemS) (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $a $c $e $r)
      (Tuple (Not $c) (And (Not $c) (Lt 3 $a)) (If (And (Not $c) (Le $a 3)) $a 0)))))
  (Map sat-three-wait (Parameters ($d Days)) (From (Tuple Bool Bool Rational))
    (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $o $t $v)
      (Tuple $o (And $o (Or $t (Lt 3 (Add $v $d))))
             (If (And $o (Not (Or $t (Lt 3 (Add $v $d))))) (Add $v $d) 0)))))
  (Statistic NagThreeIndex (Of NagThree) (Encode sat-three)
    (Step (wait $d) (sat-three-wait $d) (For $d Days)) (Step reply sat-close)
    (Step (snooze $s) (sat-keep $s) (For $s Days)) (Read nag sat-nag))
  ;; rebuilding the new index from the durable record: the record is a statistic of
  ;; the machine that observes the new index, and its read map is the rebuild
  (Machine NewIndexView (States ItemS) (Observe index sat-three) (ActionsOf ItemSnooze))
  (Map rebuild-three (From (Tuple Rational Bool Rational)) (To (Tuple Bool Bool Rational))
    (Cases ((Tuple $a $c $r)
      (Tuple (Not $c) (And (Not $c) (Lt 3 $a)) (If (And (Not $c) (Le $a 3)) $a 0)))))
  (Statistic RebuildFromRecords (Of NewIndexView) (Encode rec)
    (Step (wait $d) (rec-wait $d) (For $d Days)) (Step reply rec-reply)
    (Step (snooze $s) (rec-snooze $s) (For $s Days)) (Read index rebuild-three))
  ;; the engine index is rebuilt from the durable record in the same way
  (Machine EngineIndexView (States ItemS) (Observe index joint) (ActionsOf ItemSnooze))
  (Map rebuild-joint (From (Tuple Rational Bool Rational))
    (To (Tuple Bool Bool Rational Rational))
    (Cases ((Tuple $a $c $r)
      (Tuple $c (And (Not $c) (Lt 7 (Add $a $r)))
             (If (And (Not $c) (Le (Add $a $r) 7)) $a 0)
             (If (And (Not $c) (Le (Add $a $r) 7)) $r 0)))))
```

```
  (Statistic RebuildEngineIndex (Of EngineIndexView) (Encode rec)
    (Step (wait $d) (rec-wait $d) (For $d Days)) (Step reply rec-reply)
    (Step (snooze $s) (rec-snooze $s) (For $s Days)) (Read index rebuild-joint))

  ;; the store after the release
  (Carrier Person (Of Symbol) (All))
  (Carrier Topic  (Of Symbol) (All))
  (Carrier ItemKey (Of (Tuple Symbol Symbol)) (Product Person Topic))
  (Store ItemsS (Keys ItemKey) (Of ItemSnooze) (Global wait))
  (Keyed RecordsByKey (Of ItemsS) (Item RecordS))
  (Keyed EngineIndexS (Of ItemsS) (Item JointS))
  (Keyed NagIndexS (Of ItemsS) (Item NagThreeIndex) (View NagThree)))
;; @end

;; @excerpt wmtm-release-requests
!(dc.run WMTMRelease (CheckStatistic JointCarried (Method Symbolic)))
!(dc.run WMTMRelease (CheckNow CarriedIndex))
!(dc.run WMTMRelease (CheckStatistic RecordS (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic JointS (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic NagTwoIndex (Method Symbolic)))
!(dc.run WMTMRelease (CheckNow TwoForThree))
!(dc.run WMTMRelease (CheckStatistic NagThreeIndex (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic RebuildFromRecords (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic RebuildEngineIndex (Method Symbolic)))
!(dc.run WMTMRelease (CheckStatistic EngineIndexS (Method Lift)))
!(dc.run WMTMRelease (CheckStatistic NagIndexS (Method Lift)))
;; @end

;; ======================================== 3. receipts, monitors and the gate
;; @excerpt wmtm-receipts
(DC.Module WMTMReceipts
  (Profile Stochastic)
  (Use Decisions Laws Families Monitors Estimates)     ; PROPOSED Estimates (Part II)
  ;; one receipt per reply, released at the next cycle: was the item proposed (stale)?
  (Stream Replies (Source "streams/replies.json")
    (Digest "sha256:1f5d10eff762371dffdef5e0be4f940e39366a8fe2c6b9227bb7cdb66c82ca0f")
    (Provenance "simulated: replies to items at most seven days old, days 0-55")
    (Population "replies to items at most seven days old at the next cycle")
    (Selection "one row per reply at the next cycle, after its review, by reply time")
    (Policy "log-only phase; parser v1, v2 from day 21 10:00 to day 24 12:00"))
  (Stream RepliesBefore (Source "streams/replies_before.json")
    (Digest "sha256:3a5629468d68aba06e30587dfc23114cbf4ed6e545e426ed4d77c08caf799703")
    (Provenance "simulated: the reply receipts of days 0-21, before the parser update")
    (Population "replies to items at most seven days old at the next cycle")
    (Selection "the rows of Replies released on days 0-21")
    (Policy "log-only phase; parser v1"))
  (Stream RepliesGate (Source "streams/replies_gate.json")
    (Digest "sha256:39def219d4692915bb2d0ce2680d8a85b5aee69729e5f5c419c5c8beb01a764e")
    (Provenance "simulated: the reply receipts of days 28-55")
    (Population "replies to items at most seven days old at the next cycle")
    (Selection "the rows of Replies released on days 28-55")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
  ;; one receipt per key at its first proposal: the review's duplicate label
  (Stream KeysGate (Source "streams/keys_gate.json")
    (Digest "sha256:fd12fdce6989efac157bd53aff04b5e71b40dc710733fedfba64e8caed1ee5e8")
    (Provenance "simulated: keys first proposed on days 28-55")
    (Population "keys that the event design proposes")
    (Selection "one row per key at its first proposal, days 28-55")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
  ;; one receipt per key at the end of its seven days: did each design decide wrongly?
  (Stream LivesGate (Source "streams/lives_gate.json")
    (Digest "sha256:5eb1f54bf60f047de30f1cdfa4516dcefc3a789df5bac4763cdebe14aaaca801")
    (Provenance "simulated: keys whose daily decisions all fall on days 28-55")
    (Population "keys that are not duplicates and that either design proposed")
    (Selection "one row per key whose decisions all fall on days 28-55")
    (Policy "log-only phase; both designs; parser v1"))
  ;; one receipt per key at its first closing: was the closing note a real reply?
  (Stream ClosuresGate (Source "streams/closures_gate.json")
    (Digest "sha256:b11d52683007c2f3f2c25e367ee66f81e4e3393360e3ae9d297fd699806ff209")
    (Provenance "simulated: first closings learned between the cycles of days 27 and 55")
    (Population "first closings of keys that are not duplicates")
    (Selection "one row per key at the cycle after memory learns its first closing")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
```

```
  ;; the random audit at age four days, and the targeted audit after the alarm
  (Stream AuditRepliesGate (Source "streams/audit_replies_gate.json")
    (Digest "sha256:8f9a06bed6d5b60ee2007dde3cafd41ff198a57cdc572d7baa59251bf63b97ec")
    (Provenance "simulated: audited items with a reply by age four, days 28-55")
    (Population "items, sampled at creation with probability 1/3")
    (Selection "one row per sampled item replied by age four, audited at four")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
  (Stream AuditOpenGate (Source "streams/audit_open_gate.json")
    (Digest "sha256:840aff688d32968a5d7d2d9c15fc132ca416d109418f260686803037267e8db5")
    (Provenance "simulated: audited items without a reply by age four, days 28-55")
    (Population "items, sampled at creation with probability 1/3")
    (Selection "one row per sampled item not replied by age four, audited at four")
    (Policy "log-only phase; parser v1, journal re-parsed on day 24"))
  (Stream AuditAfterAlarm (Source "streams/audit_after_alarm.json")
    (Digest "sha256:5d6f7667bbdf76e094d5827d64beb8b36b1c5f959b40af405d1844c743d6b0fc")
    (Provenance "simulated: audited replies received after the receipts of the alarm")
    (Population "replies received from the alarm cycle to the audit on day 24")
    (Selection "every such reply, audited on day 24 at 09:00")
    (Policy "log-only phase; parser v2"))
  (Stream AuditV2 (Source "streams/audit_replies_v2.json")
    (Digest "sha256:ea80e318024f8534248b4d58a075b9243dfc1ad6dfa71d842a7bd74bfbef0038")
    (Provenance "simulated: every reply between the parser update and the audit")
    (Population "replies received from the parser update to the audit on day 24")
    (Selection "every such reply, audited on day 24 at 09:00")
    (Policy "log-only phase; parser v2"))

  ;; the monitors: a missed reply that reaches a proposal, accepted at one in twenty
  (Carrier One  (Of Symbol) (Enum one))
  (Carrier Rate (Of Symbol) (Enum accepted tenth fifth twofifths))
  (Map miss-rate (From Rate) (To Rational01)
    (Table (accepted (Rat 1 20)) (tenth (Rat 1 10)) (fifth (Rat 1 5))
           (twofifths (Rat 2 5))))
  (Map stale (Parameters ($r Rate)) (From One) (To (Law Nat))
    (Body ($u) (Coin (miss-rate $r))))
  (Monitor MissTenth (Context One) (Null (stale accepted)) (Alternative (stale tenth))
    (Columns (observation "stale")))
  (Monitor MissFifth (Context One) (Null (stale accepted)) (Alternative (stale fifth))
    (Columns (observation "stale")))
  (Monitor MissTwoFifths (Context One) (Null (stale accepted))
    (Alternative (stale twofifths)) (Columns (observation "stale"))))
;; @end

;; @excerpt wmtm-monitor-requests
!(dc.run WMTMReceipts (CheckMonitor MissTenth))
!(dc.run WMTMReceipts (CheckMonitor MissFifth))
!(dc.run WMTMReceipts (CheckMonitor MissTwoFifths))
!(dc.run WMTMReceipts
   (Detect MissTenth MissFifth MissTwoFifths (On Replies) (Level (Rat 1 100))
           (Combine Mixture)))
!(dc.run WMTMReceipts
   (Detect MissTenth MissFifth MissTwoFifths (On Replies) (Combine Mixture)
           (Scheme ShiryaevRoberts) (MeanTimeToFalseAlarm 100000)))   ; PROPOSED Scheme
!(dc.run WMTMReceipts
   (EstimateRate (On RepliesBefore) (Column "stale") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On AuditAfterAlarm) (Column "missed") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On AuditV2) (Column "missed") (Equals 1) (Level (Rat 1 100))))
;; @end

;; @excerpt wmtm-gate-requests
!(dc.run WMTMReceipts
   (EstimateRate (On RepliesGate) (Column "stale") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On KeysGate) (Column "duplicate") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On ClosuresGate) (Column "spurious") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (CompareRates (On LivesGate) (First "window_bad") (Second "event_bad") (Equals 1)
                 (Level (Rat 1 100))))
!(dc.run WMTMReceipts
   (EstimateRate (On AuditRepliesGate) (Column "missed") (Equals 1) (Level (Rat 1 100))))
!(dc.run WMTMReceipts
```

```
   (EstimateRate (On AuditOpenGate) (Column "false_close") (Equals 1)
                 (Level (Rat 1 100))))
;; @end
```

B The registry patch

The example needs one addition to the forms that Part II and its predecessors proposed: two arguments of `Detect` that select the changepoint scheme of Section 5.4. Table 11 is generated from `wx_registry_patch.json`, which redefines the request with the new arguments, makes `Level` optional because the changepoint scheme does not use it, and adds four fields to its result. Everything else the modules use is reused unchanged: Part II's patch `ls_registry_patch.json` (the libraries `RationalArith`, `Statistics` and `Estimates`, the declarations `Statistic`, `Store` and `Keyed`, and the requests `CheckStatistic`, `EstimateRate` and `CompareRates`); the `Families` library of the second of the four worked examples; and the `Laws` and `Monitors` libraries, the `Monitor` and `Stream` declarations and the `CheckMonitor` and `Detect` requests of the fourth [4, Appendix B]; Part II's patch also supplies the `View` field of `Keyed` and the `Population`, `Selection` and `Policy` fields of `Stream`. Without `wx_registry_patch.json` the static check reports three errors, all on the one `Detect` request that selects the changepoint scheme.

Entry	Name	Description
request	`Detect`	library Monitors; positional monitors (ref:Monitor, variadic); arguments On (ref:Stream, required), Level (rat01), Combine (enum:Union|Mixture, default Union), Scheme (enum:Ville|ShiryaevRoberts, default Ville), MeanTimeToFalseAlarm (nat); result Detection; statuses Estimated. Estimated. With Scheme Ville (the default): if the stream follows laws whose likelihood-ratio factors have conditional null expectation at most one, the probability that a listed monitor ever reaches its threshold is at most Level (Ville's inequality; Union gives each of k monitors the threshold k/Level, Mixture thresholds the average of the e-values at 1/Level). With Scheme ShiryaevRoberts and Combine Mixture: each monitor's Shiryaev-Roberts statistic R_n = (1 + R_{n-1}) L_n is averaged over the monitors and the alarm is raised when the average reaches MeanTimeToFalseAlarm; under the same null condition the average minus n is a supermartingale, so the expected number of observations before a false alarm is at least MeanTimeToFalseAlarm. The result names the first monitor (or the mixture) and observation to reach the threshold, or reports that none did with the largest value reached, with the stream digest, the protocol and the dependence assumptions
argument	`Detect Level`	Required with Scheme Ville: the probability bound on any false alarm
argument	`Detect Scheme`	Ville: e-processes started at the first observation, as in the fourth of the four worked examples; ShiryaevRoberts: a changepoint detector that sums e-processes started at every observation
argument	`Detect MeanTimeToFalseAlarm`	Required with Scheme ShiryaevRoberts: the alarm threshold, which bounds from below the expected number of observations before a false alarm
result fields	`Detection`	added to Detection: Scheme (enum:Ville|ShiryaevRoberts), MeanTimeToFalseAlarm (Nat?), Largest (Rational?), LargestAt (Nat?)

Entry	Name	Description
note	`Detect`	A monitor started at the first observation accumulates evidence for the null while the stream is healthy, and a change that comes late must first overcome it; the Shiryaev-Roberts scheme restarts implicitly at every observation, at the price of a guarantee on the mean time to a false alarm instead of on the probability of any false alarm

Table 11: The registry patch of the worked example.

C The frozen streams

The simulation writes twelve streams. Each is a JSON object with a schema, a provenance record and the rows, written once with the separators `,` and `:` and no other whitespace, so that its digest is reproducible. Table 12 lists them; the ten marked as pinned are declared in the module `WMTMReceipts` with their digests, and the other two are read by the oracle, the journal for the conformance replay and the proposal log for the descriptions of Section 4.1.

File	Unit	Rows	Pinned	Content
`journal.json`	note	2,326	no	the parsed journal with the ground truth of each note
`proposals.json`	proposal	3,767	no	every logged proposal of the event design with the review's labels
`replies.json`	reply	916	yes	one receipt per reply, days 0–55
`replies_before.json`	reply	362	yes	the reply receipts of days 0–21
`replies_gate.json`	reply	436	yes	the reply receipts of days 28–55
`keys_gate.json`	key	453	yes	keys first proposed on days 28–55
`lives_gate.json`	key	482	yes	keys whose daily decisions all fall on days 28–55
`closures_gate.json`	closing	451	yes	first closing of each key, released on days 28–55
`audit_replies_gate.json`	audited item	140	yes	random audit, items with a reply by age four
`audit_open_gate.json`	audited item	76	yes	random audit, items without a reply by age four
`audit_replies_v2.json`	reply	59	yes	targeted audit of the replies received since the parser update
`audit_after_alarm.json`	reply	15	yes	the replies of that audit received after the alarm's receipts

Table 12: The frozen streams, in the folder `streams/` of the bundle.

The digests, as `wx_oracle.py` recomputes them:

```
streams/journal.json
    sha256:afe7936bb50be1f0d48f68b336132124c1e0015d50540f4c7bf62b9d75594310
streams/proposals.json
    sha256:6dedf9ee36d04488dbab7da2be9629e5ce8f82d3576f77b32189aecd46cfbf2f
streams/replies.json
    sha256:1f5d10eff762371dffdef5e0be4f940e39366a8fe2c6b9227bb7cdb66c82ca0f
streams/replies_before.json
    sha256:3a5629468d68aba06e30587dfc23114cbf4ed6e545e426ed4d77c08caf799703
streams/replies_gate.json
    sha256:39def219d4692915bb2d0ce2680d8a85b5aee69729e5f5c419c5c8beb01a764e
streams/keys_gate.json
```

```
    sha256:fd12fdce6989efac157bd53aff04b5e71b40dc710733fedfba64e8caed1ee5e8
streams/lives_gate.json
    sha256:5eb1f54bf60f047de30f1cdfa4516dcefc3a789df5bac4763cdebe14aaaca801
streams/closures_gate.json
    sha256:b11d52683007c2f3f2c25e367ee66f81e4e3393360e3ae9d297fd699806ff209
streams/audit_replies_gate.json
    sha256:8f9a06bed6d5b60ee2007dde3cafd41ff198a57cdc572d7baa59251bf63b97ec
streams/audit_open_gate.json
    sha256:840aff688d32968a5d7d2d9c15fc132ca416d109418f260686803037267e8db5
streams/audit_replies_v2.json
    sha256:ea80e318024f8534248b4d58a075b9243dfc1ad6dfa71d842a7bd74bfbef0038
streams/audit_after_alarm.json
    sha256:5d6f7667bbdf76e094d5827d64beb8b36b1c5f959b40af405d1844c743d6b0fc
```

D Reproducing the results

The bundle `mettadinc_wmtm_example_package.zip` contains the module file `wmtm_example.metta` and the template `wmtm_example.template.metta` from which it is written once the stream digests are known; Part II's module file `live_memory.metta` and Part I's `memory_goals.metta`, which it imports from; the registry patch `wx_registry_patch.json` and the five patches it builds on; the static checker `ex_check.py`; the simulation `models/wx_sim.py`, the derived facts `models/wx_facts.py`, the Z3 encodings `models/wx_smt.py` and Part II's confidence sequences `models/cs.py`; the reference evaluator `ls_eval.py`; the oracle `wx_oracle.py` and its results `wx_results.json`; the script `make_module.py`; and the twelve frozen streams in `streams/`. Python 3.11 or later is assumed, with the packages numpy, scipy, z3-solver and cvc5; the results were produced with numpy 2.5.3, scipy 1.18.1, z3-solver 5.3.3 and cvc5 1.4.2.

The oracle. `python3 wx_oracle.py` runs 99 checks in about 13 seconds on one core, prints one line per check, writes `wx_results.json`, and exits with a nonzero status if any check fails. Its first part (55 checks) regenerates the simulated deployment from its seed, writes the twelve streams again into a temporary directory and compares them byte for byte with the shipped ones, and recomputes the numbers of the text: the counts of the run, the regression, its confirmation and the two monitors, a counterfactual run without the rollback, the estimates with Part II's confidence sequences, the ground truth against which they are checked, the decomposition of the stale proposals, the planning figures, and the symbolic obligations of the example's statistics in Z3 on encodings written by hand. Its second part (44 checks) reads the three module files with `ls_eval.py`, answers all 33 requests of the example from the module text, with cvc5 for the symbolic ones and from the pinned streams for the others, replays every witness by evaluation, compares each verdict and number with the first part, runs four differential tests, which are the conformance replay of Section 3.5 from the frozen journal, the randomized test of the snooze semantics, the rebuild of Section 7.5 and the replay of the journal through the release's engine index of Section 7.6, and checks that a copy of `RepliesGate` changed under its declared digest is refused. Symbolic verdicts are reported as Conditional on cvc5. The oracle checks that Z3 reaches the same verdict on every obligation and records that agreement under `agreed_by` beside each Conditional verdict in the section `verdicts` of `wx_results.json`.

The static check. The checker needs the specification's validation package:

```
MDINC_PACKAGE=/path/to/mettadinc_validation_package python3 ex_check.py \
    ex1_registry_patch.json ex2_registry_patch.json ex4_registry_patch.json \
    mg_registry_patch.json ls_registry_patch.json wx_registry_patch.json \
```

```
-- memory_goals.metta live_memory.metta wmtm_example.metta
```

The first pass reports 19 modules, 575 declarations and 162 run requests for the three files, of which 3 modules, 85 declarations and 33 run requests are the example's, all conforming to the registry with the patches. The second pass lists the diagnostics against the unpatched registry, which are exactly the forms that depend on the patches.

The reference evaluator. `ls_eval.py` is Part II's evaluator, which checks the premise of every lift, the target of every safety transfer and the digest of every stream before it computes anything, and reports each verdict with its assurance, with one extension: `Detect` accepts the `Scheme` and `MeanTimeToFalseAlarm` arguments of the patch, computes the Shiryaev–Roberts statistics of the listed monitors in exact rational arithmetic, averages them, and reports the first observation at which the average reaches the threshold; under the default scheme it also reports the largest value reached and where. It remains a test aid and not an implementation of MeTTaDinC.

The simulation. `python3 models/wx_sim.py` runs the eight weeks from the seed 20261104, prints the alarm and its confirmation, and writes the twelve streams with their digests into the directory given as its argument, by default `streams_regenerated/` beside `streams/`, so that the shipped streams are not overwritten. `python3 make_module.py` regenerates the shipped streams in place and writes `wmtm_example.metta` from the template with their digests; on an unchanged bundle it changes nothing. Every random draw of an item is made when the item is created, so the run does not depend on the order in which later quantities are computed. The ground truth stays inside the simulation and reaches the streams only as the review's labels, the audit's findings and the column `truth` of the journal, which the record designs never read.